# Réponses au manuel de certification SCE 2.0 - pili-pos-android 1.0.0

**Distributeur :** Bruno Moyse / nuagemagique.dev (Bruxelles, BE) **Produit :** pili-pos-android 1.0.0 (posSwVersion pili-pos-android/1.0.0) **Plateforme :** Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google) **Date :** À effectuer (EV-18 : date de dépôt)

Ce document parcourt linéairement le manuel `20250307_FR – Manuel de certification_POS_V11.pdf` et fournit, dans l'ordre des sections, la réponse à chaque case à cocher (☐) et à chaque champ libre. Les renvois aux annexes du dossier sont indiqués en italique.

`pili-pos-android` 1.0.0 fait l'objet d'une demande de certification initiale, distincte de celle de `pili-pos` 1.0.0 sur Linux (*Voir : CADRE_DEMANDE_FR.md*). Le moteur fiscal est le même code source dans les deux produits : les réponses fiscales (§III à §VI) reprennent celles de la demande Linux, sauf là où un élément propre à la plateforme est nommé. Une valeur qui ne peut être établie qu'au moment du dépôt (empreinte, date, mesure sur l'appareil) est signalée dans le texte par un identifiant de pièce de la forme EV-NN et remplie avant l'envoi.

Convention : - ☒ : case cochée (réponse positive ou élément présent). - ☐ : case non cochée (réponse négative ou non-applicable). - *Voir : annexe.md* : référence vers un document annexe du dossier.

---

## §I - DOCUMENTATION REQUISE

### §I.A Formulaire de demande

| Item | Réponse |
|---|---|
| ☒ Formulaire complété | Formulaire `20250115_FORMULAIRE DEMANDE_CERTIF_POS–FDM.pdf` complété avec les valeurs déclarées dans ce document. |
| ☒ Formulaire signé | Signature électronique qualifiée apposée au moment de la soumission par Bruno Moyse (distributeur). |

### §I.B Résultats de test

| Item | Réponse |
|---|---|
| ☒ Document PDF complété | Voir `RESULTATS_TESTS_UC_FR.pdf` (annexe). Couvre les UC du Basic Set + Reports avec JSON request, JSON response et ticket pour chaque UC, **à l'exclusion de M150 (event I, hors périmètre v1.0.0, cf. §III.F)**. Périmètre identique à celui de la demande Linux : 17 UC (14 du Basic Set : M110, M111, M112B, M121, M123, M130, M131, M140, M141, M160, M161, M170, M171, M172 ; 3 de Reports : M180, M182, M184 ; M150 exclu). Exécution sur l'émulateur Android, JSON sous Report_Testing/uc : À effectuer (EV-06 : résultats UC sur l'émulateur Android, requête et réponse JSON par UC). Comparaison avec le journal Linux : À effectuer (EV-07 : journaux et requêtes FDM comparés entre Android et Linux). |

## §I.C Manuels du POS

| Item | Réponse |
|---|---|
| ☒ Manuel d'utilisateur | *Voir : MANUEL_UTILISATEUR_FR.md* |
| ☒ Manuel de configuration/installation | *Voir : MANUEL_INSTALLATION_FR.md* |
| ☒ Manuel de programmation | *Voir : MANUEL_PROGRAMMATION_FR.md* |
| ☐ Manuel de service | Non applicable. Producteur = distributeur = développeur unique. Le support de service est assuré directement par le producteur (voir §I.F profil de l'entreprise). |
| ☒ Schéma d'assemblage | *Voir : SCHEMA_ASSEMBLAGE_FR.pdf* (annexe). |

## §I.D Hash SHA-1 du logiciel

| Item | Réponse |
|---|---|
| ☒ Le SHA-1 est envoyé en pièce jointe | Le SHA-1 déclaré est celui de l'APK signé `pili-pos-android` 1.0.0 : À effectuer (EV-01 : SHA-1 et SHA-256 de l'APK signé). Sont aussi donnés le SHA-1 de la bibliothèque du moteur `libpos_ffi.so` de cet APK, À effectuer (EV-02 : SHA-1 de `libpos_ffi.so`), et l'empreinte SHA-256 du certificat de signature, À effectuer (EV-03 : SHA-256 du certificat de signature). Procédure de production et de vérification : *GARANTIE_MAJ_LANGUES_FR.md* §1.2. La valeur exacte est communiquée au moment de la soumission (le hash est lié à la build finale signée). |

## §I.E Documents d'identification

**Producteur (= distributeur = développeur)**

| Champ | Valeur |
|---|---|
| Nom de l'entité juridique | Bruno Moyse (personne physique indépendante) |
| Nom commercial | nuagemagique.dev |
| Structure juridique | Personne physique (entreprise individuelle, régime franchise TVA art. 56 §2 CTVA) |
| Adresse | Rue Joseph Claes 4/1, 1060 Saint-Gilles, Belgique |
| Numéro d'identification à la TVA | BE 0783.544.917 (numéro d'entreprise BCE ; régime de franchise de TVA, art. 56 §2 CTVA) |
| Site web | https://nuagemagique.dev |
| Représentants autorisés | Bruno Moyse |
| Représentant - numéro de téléphone | +32 494 46 50 26 |
| Représentant - adresse électronique | cloud@nuagemagique.dev |
| Contact technique | Bruno Moyse |
| Contact technique - numéro de téléphone | +32 494 46 50 26 |
| Contact technique - adresse électronique | cloud@nuagemagique.dev |

**Importateurs et distributeurs**

Le producteur est également importateur et distributeur (configuration mono-acteur). Pour les déploiements clients futurs, *voir : LISTE_DISTRIBUTEURS_FR.md*.

### §I.F Profil de l'entreprise

*Voir : PROFIL_ENTREPRISE_FR.md* - couvre : - Contrôle interne (enregistrement, plugins matériels) - Organisation interne (1 développeur, internalisation 100%) - Distributeurs (dispositions contractuelles à venir pour futurs clients) - Support technique (canal email cloud@nuagemagique.dev + assistance sur site, sans accès à distance) - Analyse de risques (incl. RGPD - base légale article 6.1.c du RGPD pour l'INSZ employé) - Contrôle externe (aucun autre certificat à ce jour, pas d'assurance produit)

---

## §II - DESCRIPTION DU POS

### §II.A Identification du logiciel

| Champ | Valeur |
|---|---|
| Nom du POS | pili-pos-android |
| Modèle du POS | pili-pos-android 1.0.0 (application Android pour le terminal Sunmi D3 Pro, un seul APK) |
| N° de version du logiciel | 1.0.0 (posSwVersion pili-pos-android/1.0.0, versionCode 10000, identifiant d'application `dev.nuagemagique.pilipos`), affichée sur l'écran de connexion et sur l'écran « À propos » (*CADRE_DEMANDE_FR.md* §3) |
| Langage(s) de programmation | Rust (moteur fiscal commun avec `pili-pos`, compilé en bibliothèque native `libpos_ffi.so`) + Kotlin et Compose (interface et plateforme) |

**Versions des spécifications suivies**

| Document | Version |
|---|---|
| Description détaillée du POS | 2.2 (2026-04-17) |
| Description détaillée du FDM | 1.4 (2026-04-17) |
| Addendum 1 aux descriptions détaillées POS / FDM | 2.1 (2026-02-09) |
| Protocole FPSFIN API (FDM vers cloud SPF) | 1.10 (2026-04-14). La version 1.10 ne change rien pour le POS, qui ne parle jamais à ce protocole : elle ajoute des précisions sur le drapeau ACTIVE et sur le FDM de secours, et abaisse la limite du service d'enregistrement de 1 Mo à 0,5 Mo (côté FDM). |
| Jeux de tests Basic Set | 1.8 (2026-08-21) |
| Jeux de tests Advanced Set | 1.6 (2026-08-21) |
| Jeux de tests Reports | 1.3 (2026-08-21) |
| FAQ producteurs | 2026-07-17 |
| Manuel de certification | 1.1 |
| Procédure de certification (introduction) | 2026-09-03 |

### §II.B Déclaration d'accès à la base de données

| Item | Réponse |
|---|---|
| ☒ Les utilisateurs n'ont accès qu'en lecture seule | La table `fiscal_events` (archive fiscale) est append-only : des déclencheurs SQLite (migration 4) refusent tout DELETE et tout UPDATE, et aucune commande du moteur n'en émet (vérifié par grep hors modules de test et par test d'intégration). Pour les autres tables, les rôles utilisateur (Admin/Cashier/Waiter) gouvernent les opérations métier autorisées via la couche de commandes. Aucun utilisateur du POS n'a d'accès SQL direct à la base : le terminal n'a ni console, ni explorateur de fichiers, ni compte de contrôle, et le droit de l'opérateur se limite à la lecture sous forme d'export (*SECURITE_BD_FR.md* §8.1). *Voir : SECURITE_BD_FR.md et DECLARATION_FONCTIONS_INTERDITES_FR.md*. |

## §II.C Caractéristiques de la base de données

| Champ | Valeur |
|---|---|
| La base de données est | ☒ Locale (SQLite embarqué) |
| La base de données et le POS sont tous les deux basés sur le cloud | ☐ Non |
| Identification du DBMS | SQLite |
| N° de version de la base de données | SQLite 3.45+ (version embarquée par la bibliothèque du moteur, crate `rusqlite`) |
| Nom du logiciel et version | pili-pos-android 1.0.0 (le DBMS est embarqué dans la bibliothèque `libpos_ffi.so` de l'APK) |
| Chemin d'accès | `pos.db` dans `files/engine/`, stockage interne privé de l'application, jamais sur la carte SD (*PLATEFORME_ANDROID_FR.md* §5) |

**Aperçu des tables**

| Item | Réponse |
|---|---|
| ☒ La liste contient toutes les colonnes demandées | *Voir : SCHEMA_BD_FR.md* - inventaire complet par table avec nom, champ, type, longueur, description. |
| ☒ La description est brève et claire | Idem. |

**Informations supplémentaires SPF Finances**

| Item | Réponse |
|---|---|
| ☒ Description du format de la base de données | *Voir : SCHEMA_BD_FR.md* |
| ☒ Sécurité de la base de données | *Voir : SECURITE_BD_FR.md* (WAL, FK constraints, append-only enforcement, stockage privé du bac à sable Android, terminal verrouillé par le propriétaire de l'appareil, signature ECDSA du FDM conservée, contrôle de cohérence de chaque réponse FDM, shortSignature SHA-1) |

| Item | Réponse |
|---|---|
| ☒ Données utilisées et créées par le POS | *Voir : FEUILLE_ACQUISITION_DONNEES_FR.md* |
| ☒ Aperçu des tables contenant les données des events | Table `fiscal_events` documentée dans `SCHEMA_BD_FR.md`. Table append-only avec 1 ligne par événement N/P/T/C/I/F/S/R. |
| ☒ Disposition des tables de données | *Voir : SCHEMA_BD_FR.md* (organisation par domaine : menu, orders, payments, fiscal, users, reports) |
| ☒ Relations entre les tables | *Voir : SCHEMA_BD_FR.md et SCHEMA_ER_FR.pdf* (diagramme ER en annexe) |

**Captation de la base de données**

| Item | Réponse |
|---|---|
| ☒ Les agents SPF peuvent accéder/copier les données pertinentes | Procédure documentée dans `FEUILLE_ACQUISITION_DONNEES_FR.md` et `SECURITE_BD_FR.md` §8 (art. 16 et 102) : le Gérant ouvre l'écran « Export pour le SPF » des réglages et exporte vers une clé USB ou une carte SD. Le dossier exporté contient une copie cohérente de `pos.db` (lecture seule pour l'auditeur, qui l'ouvre sur son poste), les 7 sauvegardes les plus récentes, les journaux, le journal de la plateforme, un fichier d'identité et `MANIFEST.sha256` pour contrôler l'intégrité. |
| ☒ URL, nom d'utilisateur et mot de passe | Aucun compte SPF ni mot de passe système : le terminal n'a ni console ni SSH (kiosque verrouillé, *SECURITE_BD_FR.md* §3.3 et §8.4). Au rendez-vous d'admission, l'agent SPF accède aux données **sur place** : il ouvre la session du profil Gérant de la machine de test (§III, Utilisateurs & Mots de Passe) pour lancer l'export, puis ouvre la copie de `pos.db` sur son propre ordinateur. Aucun accès distant à la base n'est possible. |
| ☐ Les données exportées doivent être converties | Non. SQLite est lisible directement par tout outil standard (DB Browser for SQLite, sqlite3 CLI, etc.). Aucune conversion nécessaire. |
| ☐ Les scripts/programmes de conversion sont fournis | Non applicable (cf. ligne précédente). |

**Sécurité de la base de données**

| Item | Réponse |
|---|---|
| ☒ Audit trail possible (timestamps, utilisateurs, Primary Key, …) | Chaque ligne `fiscal_events` porte `created_at` (timestamp UTC), `user_insz` (identité opérateur), `id` (PK auto-incrément monotone), `sequence_number` (compteur fiscal séquentiel), `pos_id` et `terminal_id`. Toute autre table porte `created_at`. *Voir : SCHEMA_BD_FR.md*. |

| Item | Réponse |
|---|---|
| Mesures contre la manipulation frauduleuse | *Voir : SECURITE_BD_FR.md* §1 “Append-only enforcement” + `DECLARATION_FONCTIONS_INTERDITES_FR.md`. Résumé : pas de DELETE/UPDATE applicatif sur `fiscal_events` (vérifié par grep hors modules de test) et des déclencheurs SQLite qui les refusent, stockage privé du bac à sable Android (aucune permission de stockage, une seule application accède aux données), terminal verrouillé (propriétaire de l’appareil, mode Lock Task, débogage coupé en production), SHA-1 de l’APK vérifiable sur la caisse (écran « À propos »), build reproductible (Cargo.lock, rust-toolchain.toml, image de build épinglée par empreinte, deux builds comparées par `scripts/android/check_repro.sh`), aucun port en écoute ni assistance à distance côté POS, FDM = seul canal sortant par défaut. |

## §II.D Garantie du produit

**Garantie que machine-test et SCE installé chez le client sont identiques :**

L’identité bit-à-bit entre la build cert soumise au SPF Finances et la build déployée chez chaque client est garantie par :

1. **Build reproductible.** `Cargo.lock` et `android/rust/Cargo.lock` sont commités dans le dépôt. `rust-toolchain.toml` épingle la version exacte du compilateur (Rust 1.99.0), et Gradle et ses dépendances sont épinglés par empreinte. L’APK non signé est construit dans un conteneur dont l’image est désignée par son empreinte (`android/docker/`), et la commande de contrôle est documentée et figée :

   `scripts/android/check_repro.sh`

   (deux builds du même commit dans deux conteneurs sans réseau, comparées octet par octet. Seule la build en conteneur est la référence. Empreinte de l’image et SHA-1 des deux APK : À effectuer (EV-05 : empreinte de l’image de build et SHA-1 des deux APK non signés identiques).)

2. **Empreinte SHA-1 de l’APK.** L’APK est signé après la vérification de reproductibilité. Le SHA-1 de l’APK signé, déclaré au formulaire §I.D, se lit sur toute caisse installée dans l’écran « À propos » (avec celui de `libpos_ffi.so` et celui du certificat de signature) et se recalcule sans la caisse sur l’artefact public déclaré. *Voir : GARANTIE_MAJ_LANGUES_FR.md* §1.2.

3. **Terminal verrouillé.** Le déploiement client se fait par le producteur, qui installe l’APK signé sur un terminal mis sous propriétaire de l’appareil, sans compte Google (mode Lock Task, installations depuis des sources inconnues interdites, débogage coupé en production). Après la mise en service, le seul chemin d’arrivée de l’APK fiscal est l’outil de mise à jour déclarée, qui refuse tout APK dont l’empreinte ou le certificat diffèrent de la déclaration. *Voir : PLATEFORME_ANDROID_FR.md* §3 et *SECURITE_BD_FR.md* §3.

4. **Tag git.** Le commit de la build cert est tagué `android-v1.0.0` ; le moteur fiscal commun est celui de l’étiquette de gel `linux-v1.0.0-rc2` (puis de l’étiquette finale) : À effectuer (EV-04 : commit de l’étiquette du moteur). Toute MAJ ultérieure passe par un nouveau cycle de certification (procédure §II.E + Article 109 du Manuel).

## §II.E Mises à jour

**Procédure de mise à jour du POS :**

Toute modification du logiciel POS post-certification nécessite une nouvelle demande de certification (mise à jour). Sur la machine de test, la nouvelle version est remise sur clé USB (les trois fichiers de la déclaration : `declared.json`, sa signature minisign et l'APK), puis installée sur place par le Gérant (« Config » > « Mise à jour » > « Depuis une clé USB »). L'outil refuse tout APK dont la signature de la déclaration, les empreintes SHA-1 et SHA-256, le code de version (qui doit être strictement supérieur) ou le certificat de signature ne correspondent pas à la déclaration. Aucune mise à jour automatique en arrière-plan n'est implémentée : c'est une décision explicite de scellement. *Voir : GARANTIE_MAJ_LANGUES_FR.md* §2.

**Correctifs urgents.** La procédure du SPF Finances (Introduction à la certification, 2026-09-03, §4) prévoit : « Les mises à jour ayant un impact sur le volet SCE ne peuvent être installées chez les clients qu'après avoir été certifiées par nos soins. Les corrections urgentes de bugs ou les solutions à des blocages chez des clients constituent une exception à cette règle, mais doivent tout de même nous être signalées. » pili-pos-android s'y conforme : un correctif urgent peut être installé chez un client avant la nouvelle certification, à condition d'être signalé au SPF Finances, et il figure dans la demande de mise à jour suivante ; il suit le même chemin que toute mise à jour (déclaré, puis installé par l'outil de mise à jour). Dans la base, les versions de migrations 12 à 19 sont réservées à ces correctifs certifiés (`crates/pos-db/src/migrate.rs`) ; l'intégration HubRise commence à la version 20.

**Solution d'accès à distance pour l'agent SPF :**

Aucune solution d'accès à distance, ni permanente ni à la demande, n'existe sur `pili-pos-android` : ni sur la machine de test livrée à l'Intake, ni chez le client (la spec SCE 2.0 demande un appliance scellé, sans SSH ni service de gestion réseau, et aucun programme de prise de main à distance n'est installé). Pour les interventions techniques entre la soumission et le rendez-vous d'admission, comme en production :

- **Accès local au rendez-vous d'admission.** Lors de l'Intake, l'agent SPF accède physiquement à l'écran de la machine de test. Il n'y a pas de login système : l'agent utilise les écrans du Gérant protégés par PIN (profil du §III, Utilisateurs & Mots de Passe) et l'export vers une clé USB ou une carte SD (*SECURITE_BD_FR.md* §8, *FEUILLE_ACQUISITION_DONNEES_FR.md*). C'est suffisant pour vérifier la version (écran « À propos »), exécuter les tests et copier la DB.
- **Assistance sur place.** Le producteur se déplace : écrans du Gérant, lot de provisionnement signé sur clé USB, export des journaux et des sauvegardes. Le débogage USB est coupé en production et aucun port n'écoute sur le terminal (*PLATEFORME_ANDROID_FR.md* §7). *Voir : GARANTIE_MAJ_LANGUES_FR.md* §2.2.
- **Mises à jour du système Android.** Elles sont distinctes de celles de l'application et ne changent jamais l'APK déclaré. Comme construit, le propriétaire de l'appareil applique une fenêtre hors des heures de service (de 03:00 à 05:00) ; politique relevée sur le profil émulé avec services Google le 15.10.2026 (EV-13, `evidence/gms/EV13_2026-10-15.md`).

Cette absence de solution d'accès à distance permanente est délibérée : pour un terminal de caisse fiscale, ouvrir un canal d'administration distante augmente la surface d'attaque sans bénéfice opérationnel (le producteur n'a pas accès à la caisse du client en service).

## §II.F Matériel

**Tableau du matériel - machine-test (Intake) + production client**

| Type | Délivré machine-test à l'Intake | Production client | Description |
|---|---|---|---|
| **Ordinateur(s)** | | | |

| Type | Délivré machine-test à l'Intake | Production client | Description |
|---|---|---|---|
| POS (kiosk) | ☒ | ☒ | Sunmi D3 Pro, variante avec services Google (Android 13, arm64, écran tactile intégré 1920×1080 en paysage), provisionné sans compte Google. Tel que construit, la boutique Play et les applications Google destinées à l'utilisateur sont masquées (onze paquets présents et masqués sur le profil émulé, EV-13, `evidence/gms/EV13_2026-10-15.md`) et le mode kiosque suspend les autres écrans d'accueil et les Réglages ; Play Protect sur un vrai terminal : À effectuer (EV-15 : Play Protect sur le D3 Pro réel). Machine de test : profil émulé `sunmi_d3_pro_gms_api33`, sauf si un vrai D3 Pro est disponible, À effectuer (EV-15 : fiche d'un vrai D3 Pro, ou déclaration du profil émulé). *Voir : PLATE-FORME_ANDROID_FR.md* §1 et §3.7 |
| Ordinateur fixe | ☐ | ☐ | Le POS est un kiosk dédié |
| Ordinateur portable | ☐ | ☐ | |
| Tablette | ☐ | ☐ | |
| Kiosk | ☒ | ☒ | ☒ Le POS est seulement un kiosk |

| Type | Délivré machine-test à l'Intake | Production client | Description |
|---|---|---|---|
| **FDM** | ☐ | ☒ | FDM module2 (PRACSYS, certifié SCE 2.0) chez le client. Pas de boîtier livré à l'Intake : la machine de test signe auprès du simulateur en ligne module2 (`https://fdm.module2.be/graphql/` HTTP possible, jeton Bearer, posId `CPOS0031234567`, terminal `POS-2-ANDROID`) ; méthode de connexion et ports documentés dans *SCHEMA_ASSEMBLAGE_FR.md* §7. |
| **Multiprise** | ☒ | ☐ | Fourni à l'Intake pour faciliter le branchement physique |
| **Imprimante thermique** | ☒ | ☒ | Epson TM-m30III (80 mm), raccordée en LAN (TCP 9100) ; RJ11 pour le tiroir-caisse. Comportement d'une Epson réelle avec le terminal : À effectuer (EV-15 : imprimante Epson réelle) |
| **Connectivité** | | | |
| Router/Hub | ☒ | ☐ | Mini-routeur fourni à l'Intake. En production, le restaurant fournit son propre réseau. |
| Câbles | ☒ | ☐ | Câbles RJ45 (routeur, FDM, imprimante), alimentation du terminal, clé USB du lot de provisionnement. En production, fournis avec l'install. |
| Clé Wifi | ☐ | ☐ | Pas de clé : le Wi-Fi est intégré au terminal et réglable sur place par le Gérant. Liaison filaire seulement si le modèle en offre une : À effectuer (EV-15 : liaison filaire d'un vrai D3 Pro) |
| Moxa | ☐ | ☐ | Non utilisé |
| Raspberry Pi | ☐ | ☐ | Non utilisé |
| **Périphériques d'entrée** | | | |

| Type | Délivré machine-test à l'Intake | Production client | Description |
|---|---|---|---|
| Souris | ☐ | ☐ | Interface tactile uniquement |
| Clavier | ☐ | ☐ | Idem (clavier virtuel à l'écran de l'application si nécessaire) |
| Scanner de produits (client) | ☐ | ☐ | Non supporté |
| Lecteur de carte | ☐ | ☐ | Paiement carte via terminal TPE externe (non intégré) |
| Clés Dallas | ☐ | ☐ | Auth via INSZ + PIN, pas de support clés Dallas ; un badge facultatif (NFC ou lecteur USB), qui n'est pas une clé Dallas, peut remplacer le PIN des Serveurs et des Apprentis, jamais celui d'un Gérant (§III.E « Pointage par badge ») |
| Périphérique supplémentaire | ☐ | ☐ | Aucun périphérique permanent. Clé USB ou carte SD seulement ponctuellement (lot de provisionnement, mise à jour, export SPF) |
| **Périphériques de sortie** | | | |
| Écran utilisateur (client-facing) | ☐ | ☐ | Pas en MVP |
| Écran pour la cuisine/le bar | ☐ | ☐ | Sortie cuisine via imprimante dédiée (kitchen printer), pas KDS |
| Balance | ☐ | ☐ | Pas de prix au poids |
| **Livraison** | ☐ | ☒ | Commandes de livraison reçues des plateformes et du webshop, signées par le FDM (N, ou P puis N à la remise). *Voir : COMMANDES_EN_LIGNE_FR.md* |
| Distributeur de nourriture | ☐ | ☐ | |
| Robot | ☐ | ☐ | |
| **Commande à distance** | ☐ | ☐ | |

| Type | Délivré machine-test à l'Intake | Production client | Description |
|---|---|---|---|
| Boutique en ligne | ☐ | ☒ | Webshop de l'établissement relié par HubRise : chaque commande se termine par un N (FAQ Q203), paiement `ONLINE` si payée en ligne. Présent dans l'APK certifié, activé par installation (coupé par défaut, donc coupé sur la machine de test). *Voir : COMMANDES_EN_LIGNE_FR.md* |
| Commande en ligne | ☐ | ☒ | Idem, pour les plateformes de livraison (paiement `APP`) |
| Scanner de code QR | ☐ | ☐ | |
| **Facturation/paiement** | | | |
| App de commande avec facturation | ☐ | ☐ | |
| Processus via site internet | ☐ | ☐ | |
| Tiroir-caisse connecté | ☒ | ☒ | Tiroir RJ11 raccordé à l'imprimante, ouverture par séquence ESC/POS |
| **Autres** | ☐ | ☐ | |

### §II.G Schéma d'assemblage

| Item | Réponse |
|---|---|
| ☒ Schéma d'assemblage fourni en annexe | *Voir : SCHEMA_ASSEMBLAGE_FR.pdf* (annexe). Représente : POS (1) – Wi-Fi ou RJ45 –> Routeur (2) ; Routeur (2) – Ethernet –> FDM (3) ; Routeur (2) – Ethernet –> Imprimante (4) – RJ11 –> Tiroir-caisse (5), le POS parlant à l'imprimante et au FDM par le routeur. Liaison filaire du terminal : À effectuer (EV-15 : liaison filaire d'un vrai D3 Pro). |

Remarques point I (matériel) : aucune option exotique. Configuration mono-terminal, mono-FDM, mono-imprimante, mono-tiroir.

---

## §III - FONCTIONNALITÉS DU POS

### Configuration du POS - A. Paramètres d'installation

*Voir : MANUEL_INSTALLATION_FR.md*

| Item | Réponse |
|---|---|
| ☒ La liste est claire et complète | Voir le fichier de configuration documenté `pos.json` avec les champs obligatoires (`fdm_url`, `printer_address`, `restaurant_est_no`, `pos_id`, `terminal_id`, etc.), livré dans le lot de provisionnement signé (*MANUEL_INSTALLATION_FR.md*). |
| ☒ Les paramètres définis sont indiqués | Idem. |
| ☒ Les paramètres définis peuvent être fixes | Oui. `pos.json` est dans le lot de provisionnement importé depuis une clé USB, dont la signature minisign est vérifiée avant tout usage. Il est copié dans le stockage privé de l'application (`files/provisioning/bundle/`), hors de portée des autres applications et de l'opérateur, et ne change que par l'import d'un nouveau lot signé. |
| Logs stockés | ☒ Localement (le POS est un appareil mono-terminal, pas de serveur de logs distant) |
| ☒ Les logs ne peuvent pas être supprimés | Les logs applicatifs sont des fichiers `files/logs/pili-pos.log.AAAA-MM-JJ` (un par jour) du stockage privé de l'application, écrits par le POS lui-même (`tracing-appender`). Aucune commande du POS ne les efface et l'opérateur n'a ni shell ni explorateur de fichiers ; seule la rotation technique supprime le plus ancien au-delà de 90 fichiers (*FEUILLE_ACQUISITION_DONNEES_FR.md* §3). Ces logs ne sont pas la source fiscale : les données fiscales sont dans la table `fiscal_events`, protégée par des déclencheurs SQLite qui refusent tout DELETE et tout UPDATE. *Voir : FEUILLE_ACQUISITION_DONNEES_FR.md* §1.3 et *SECURITE_BD_FR.md*. |

## Configuration du POS - B. Configuration Internet

| Item | Réponse |
|---|---|
| Le POS nécessite une connexion Internet | ☐ Non par défaut (le POS communique uniquement avec le FDM sur le LAN local). Toutefois, le FDM lui-même nécessite Internet pour communiquer avec SPF Finances. Quatre connexions sortantes HTTPS (443) sont possibles, chacune seulement quand elle est activée ou lancée : (a) si le ticket numérique par e-mail est configuré (`pos.json::digital_ticket`, §III.H), vers `api.scaleway.com` pour l'envoi des e-mails ; (b) si les commandes en ligne sont activées (`pos.json::hubrise_enabled`, coupé par défaut donc coupé sur la machine de test, §III.M), vers `api.hubrise.com` (commandes reçues, statuts, catalogue, ventes de la caisse sans données de client) et, le temps de l'autorisation du compte HubRise par le Gérant (OAuth), vers `manager.hubrise.com` ; (c) si l'export nocturne est configuré (section `export` de `pos.json`, absente par défaut donc absente sur la machine de test), vers le point de réception propre à l'installation (`export.url`), uniquement pour y envoyer une sauvegarde chiffrée de bout en bout et une copie statistique sans donnée directement identifiante, après chaque rapport Z réel (*EXPORT_NOCTURNE_FR.md*) ; (d) quand un Gérant lance la mise à jour de l'application, vers le registre public des artefacts déclarés, par un tirage anonyme (*GARANTIE_MAJ_LANGUES_FR.md* §2.1). La connexion (c) n'est jamais nécessaire à une vente, à un Z ni à l'accès aux données par le SPF Finances : coupée ou en panne, la caisse fonctionne à l'identique. Sans ces configurations ni mise à jour lancée, aucun trafic sortant n'est initié par le POS lui-même au-delà du FDM. Les racines de confiance TLS sont embarquées dans l'APK (*PLATEFORME_ANDROID_FR.md* §6). |
| La connexion Internet peut se faire par | ☒ Wi-Fi (réglable sur place par le Gérant) - ☐ Ethernet : seulement si le modèle du terminal offre une liaison filaire, À effectuer (EV-15 : liaison filaire d'un vrai D3 Pro) |

| | |
|---|---|
| Besoins spécifiques pour la mise en place du réseau | IP statique pour le FDM (réservation DHCP recommandée). Aucun port entrant à ouvrir sur le POS. Le POS ouvre une connexion sortante TCP vers l'IP/port GraphQL du FDM, et, seulement si elles sont activées, des connexions sortantes HTTPS (443) vers `api.scaleway.com` (ticket numérique configuré) et vers `api.hubrise.com` (`hubrise_enabled`, plus `manager.hubrise.com` pour l'autorisation OAuth) et vers le point de réception de l'export nocturne (section `export` configurée ; l'hôte est propre à l'installation, port 443 ou le port indiqué dans `export.url`). Le registre des artefacts déclarés (mise à jour, §II.E) n'est joint que quand un Gérant lance la mise à jour, jamais en arrière-plan ; aucun outil d'accès distant n'est installé. |

## Configuration du POS - C. Utilisateurs & Mots de Passe

**Tableau utilisateurs**

| Rôle | Nom d'utilisateur | Mot de passe | Commentaires |
|---|---|---|---|
| Compte système (console) | Aucun | Aucun | Pas de login système : le terminal démarre directement l'application en plein écran (mode kiosque), sans console ni compte `spf` (*SECURITE_BD_FR.md* §8.4). L'agent SPF utilise le profil Gérant ci-dessous. |
| POS login - Gérant | INSZ : `94060242576` (test) | PIN : `1111` | Rôle `admin` (= Gérant) : accès complet, dont rapports, copies, remboursements, mouvements de caisse et mode formation. Voir `PR0–FILS_UTILISATEURS_FR.md`. |
| POS login - Serveur (Elisa) | INSZ : `84022899837` (test) | PIN : `2222` | Rôle `cashier` (= Serveur) : commandes, addition et encaissement. Elisa, chef de salle du jeu de test SPF (UC_TestSetup). |
| POS login - Serveur (John) | INSZ : `75061189702` (test) | PIN : `4444` | Rôle `cashier` : John, barman du jeu de test SPF. |
| POS login - Serveur (Sammy) | INSZ : `80113078968` (test) | PIN : `5555` | Rôle `cashier` : Sammy, sommelier du jeu de test SPF. |
| POS login - Apprenti | INSZ : `99012345667` (test) | PIN : `3333` | Rôle `waiter` (= Apprenti) : commandes uniquement, sans encaissement. |

| Rôle | Nom d'utilisateur | Mot de passe | Commentaires |
|---|---|---|---|
| POS login - Technicien | INSZ : `00000000097` | PIN : `0097` | NISS technicien standard SPF, rôle `admin`. |
| Identité système (robot) | INSZ : `00000000029` | Aucune connexion possible | NISS robot/automate standard SPF, utilisé comme opérateur du rapport Z automatique (FAQ Q501). Elle ouvre aussi la fiche d'une commande en ligne reçue ; les événements de cette commande sont signés au nom de l'employé qui l'accepte. Ce n'est pas un compte de connexion : il n'a pas de tuile et la caisse refuse de l'ouvrir en session. |

Ces comptes sont créés par la graine de test (`src-tauri/seed_spf_test.sql`, fournie comme `seed.sql` du lot de provisionnement de la machine de test et appliquée seulement à une base qui n'existe pas encore) ; le test `access_control.rs::the_admission_seed_logs_in_every_documented_profile` se connecte avec chacun des six comptes de connexion, vérifie son rôle, et vérifie que l'identité système est refusée. Un Gérant doit toujours avoir un PIN. Au tout premier démarrage seulement (aucun Gérant n'a encore de PIN), un Gérant sans PIN choisit le sien à sa première connexion (`commands/auth.rs::do_login_with_new_pin`, `MANUEL_UTILISATEUR_FR.md` §2.1). Ensuite, un nouveau Gérant est créé avec son PIN, confirmé par le PIN du Gérant qui le crée (`commands/auth.rs::do_create_user`) ; un Gérant sans PIN ne peut plus choisir lui-même le sien (code `first_pin_needs_manager`).

**Mode de connexion :** le POS s'ouvre par INSZ + PIN. Pour les Serveurs et les Apprentis, un badge (carte ou porte-clés) peut remplacer le PIN sur l'écran de connexion si le Gérant a activé « Lecteur de badges » et « Connexion par badge » (réglages désactivés par défaut) ; le badge est lié à l'employé par le Gérant avec son PIN (`commands::badges::do_link_badge`, §III.E « Pointage par badge »). Les Gérants se connectent **toujours avec leur PIN** : leur badge ne fait qu'ouvrir leur clavier de PIN. Un employé peut refuser le badge : le PIN et l'enregistrement de sa présence par un Gérant restent disponibles.

**Gestion des utilisateurs et rôles**

| Item | Réponse |
|---|---|
| ☒ Description de la gestion des utilisateurs et des rôles | *Voir : PROFILS_UTILISATEURS_FR.md* (mapping `admin`/`cashier`/`waiter` <-> Gérant/Serveur/Apprenti) + `MANUEL_PROGRAMMATION_FR.md` §Users (templates SQL d'insertion). |
| ☒ Description des droits d'accès | *Voir : PROFILS_UTILISATEURS_FR.md* - matrice fonctionnalité × rôle. |

Stockage des données utilisateur : table `users` (`insz` PK 11 chiffres, `name`, `role`, `pin_hash`, `active`). Droits d'accès : la liste des utilisateurs actifs est lue par l'écran de connexion (choix de l'opérateur) ; la création d'utilisateurs est réservée au rôle `admin` ; un PIN est modifié par son titulaire ou par un `admin` ; suppression interdite (active=0 à la place). Les droits sont vérifiés par le backend pour la session ouverte par la connexion

(`crates/pos-engine/src/commands/access.rs`), pas seulement par l'interface. Audit trail via timestamps + journal applicatif.

**Verrouillage du PIN :** après 5 PIN erronés d'affilée pour un utilisateur, toute tentative (connexion ou confirmation par PIN, même avec le bon PIN) est refusée pendant 30 s ; la durée double à chaque nouvel échec, jusqu'à 15 min ; un PIN correct remet le compteur à zéro. Les compteurs sont en mémoire : un redémarrage de la caisse les remet à zéro (`crates/pos-engine/src/commands/access.rs`).

## Connexion avec le FDM - A. Connexion du POS au FDM

| Mode | Réponse |
|---|---|
| Autonome | ☐ |
| ☒ LAN | Le POS et le FDM sont sur le même LAN. Connexion HTTP(S) GraphQL. |
| Basée sur le cloud | ☐ |
| Configurable | ☐ |

| Options | Réponse |
|---|---|
| Load-balancing | ☐ Non (un seul FDM par POS) |
| Si OUI, middleware ? | N/A |
| POS compatible avec d'autres marques FDM | ☒ Oui par conception (protocole GraphQL standard SPF, aucune extension propriétaire). FDM de référence testé : module2 (PRACSYS), via son simulateur en ligne. Autres marques certifiées FDM 2.0 : compatibles par protocole, non testées. |
| ☒ Les terminaux ne peuvent pas communiquer avec le FDM à moins d'être identifiés avec posId | Confirmé. Chaque requête sortante du POS porte le champ `posId` 14 caractères dans le payload GraphQL (vérifié dans `crates/pos-fiscal/src/inputs.rs::CommonFields`). |

## Connexion avec le FDM - B. Connexion au FDM

| Item | Réponse |
|---|---|
| ☒ La connexion au POS est expliquée | *Voir : DOCUMENT_PROCESSUS_TECHNIQUE_FR.md* §3.1 Transport et §9 Carte des sources. La connexion utilise `reqwest` (client HTTP Rust) sur l'URL `fdm_url` configurée dans `pos.json` (lot de provisionnement signé). |
| Logiciel intermédiaire externe | ☒ Aucun (le POS parle GraphQL directement au FDM, pas de proxy intermédiaire) |

## Connexion avec le FDM - C. FDM Non-Opérationnel

| Item | Réponse |
|---|---|
| ☒ La déconnexion du FDM est expliquée | *Voir : STRATEGIE_COMPTEURS_FR.md* §4 “Blocage opérationnel sur déconnexion” et *DOCUMENT_PROCESSUS_TECHNIQUE_FR.md* §5. Résumé : (a) le POS détecte l’erreur **immédiatement** au prochain appel `FdmClient::status` ou `sign_*` (pas seulement quand une transaction échoue) ; (b) un voile bloquant s’affiche à l’écran, le checkout est refusé tant que le FDM n’est pas opérationnel ; (c) si le FDM est injoignable ou refuse la requête, rien n’est archivé et le numéro de ticket reste libre. Si une requête a été envoyée mais que la réponse est perdue (délai dépassé, connexion coupée, HTTP 500, réponse illisible), la requête exacte reste stockée (`pending_signs`, statut ouvert) : avant toute autre signature, le POS la renvoie **octet pour octet**, et le FDM, s’il l’avait signée, renvoie sa réponse avec `DUPLICATE_REQUEST` (règle des doublons de la spec) : l’événement est alors archivé une seule fois, sous son premier numéro, avec ses effets. Tant que la réponse n’est pas revenue, l’opérateur voit « La caisse attend la réponse de la boîte noire pour l’opération précédente. Vérifiez la connexion et réessayez. » (code `FDM_ANSWER_PENDING`) ; la tâche de fond renvoie la requête toutes les 60 s et la caisse se débloque seule. Il n’y a pas d’abandon automatique : sans réponse, la signature reste bloquée (même principe que « pas de FDM, pas de vente »). Mécanisme applicatif : `crates/pos-engine/src/commands/signing.rs` est le **seul** chemin de signature et d’archivage ; une transaction SQLite avance le compteur, archive l’événement (`SignResult` valide uniquement) et clôt la ligne `pending_signs`. Toute opération de signature s’exécute sous un verrou unique (pas de numéro partagé, un Z attend une vente en cours) ; une action de l’écran attend ce verrou 30 s au plus (code `signing_busy`). Voir STRATEGIE_COMPTEURS_FR.md §3 et §5. |

## §III.A Events possibles

| Event | Disponible ? | Imprimable papier ? | Imprimable digitalement ? | Copie possible (event C) ? |
|---|---|---|---|---|
| N - Sales | Obligatoire (☒) | Obligatoire (☒) | ☒ (e-mail, `checkout::checkout` - voir §III.H) | ☒ (via M160 signCopy) |
| R - Report | Obligatoire (☒) | ☒ | ☐ (pas câblé - pas de ticket client à envoyer) | ☐ (ne s’applique pas aux R) |
| S - Social | Obligatoire (☒) | N/A | N/A | N/A |

| Event | Disponible ? | Imprimable papier ? | Imprimable digitalement ? | Copie possible (event C) ? |
|---|---|---|---|---|
| P - Pro Forma | ☒ | ☒ | ☐ (pas câblé sur `sign_table_order/sign_copy_bill` en v1.0.0) | ☒ (via M161 signCopy_Order) |
| T - Mode Training | ☒ | ☒ (avec watermark TRAINING) | ☒ (même commande `checkout::checkout`, mentions TRAINING conservées) | ☒ (via M172 signCopy_Training) |
| I - Invoice | ☐ non disponible en v1.0.0 (mise à jour certifiée ultérieure) | ☐ | ☐ | ☐ |
| C - Copy | ☒ (M160/M161/M172) | ☒ | ☒ (e-mail, `copy::copy_ticket` - sans lien/QR, le FDM ne renvoie pas de `verificationUrl` pour C) | Interdit (par la spec) |
| F - Financial | ☒ (M130 money_in_out, M131 drawer_open) | ☐ (events F sont non-imprimés en standard) | ☐ | ☐ |

Le remboursement (M111/M112B) est lui aussi un événement **N** et suit la même colonne “Imprimable digitalement” que la vente (même choix de support, `refund::refund_checkout`). Chaque ligne remboursée est valorisée au prix de la ligne de vente d’origine qu’elle désigne ; si le même produit a été vendu à plusieurs prix et que la ligne n’est pas désignée, le remboursement est refusé (code `refund_line_ambiguous`, `crates/pos-engine/src/commands/refund.rs`).

## §III.B Numérotation d’events

| Item | Réponse |
|---|---|
| PosFiscalTicketNo | ☒ Par type d’event - *Voir : STRATEGIE_COMPTEURS_FR.md* §1 “stratégie #1 de la spec POS v2.2 §385-388”. |
| Numérotation des tickets | ☒ Par type d’event (mêmes compteurs `fiscal_counters` que PosFiscalTicketNo) |

## §III.C Fonctionnalités interdites

| Item | Réponse |
|---|---|
| ☒ Le producteur déclare que de telles fonctionnalités (suppression, modification, ajout d'enregistrements déjà enregistrés) ne sont pas disponibles | Déclaration signée *DECLARATION_FONCTIONS_INTERDITES_FR.md* couvrant : pas de DELETE/UPDATE sur fiscal_events, pas d'antidate, pas de manipulation de compteur, pas de bypass FDM, pas d'effacement de logs, pas de réimpression silencieuse, pas de tickets sans QR, pas de multi-user non-traçable, pas de communication directe avec SPF Finances. |

## §III.D Fonctions obligatoires

**Enregistrement zéro :**

Un enregistrement **sans ligne de produit** ne peut **techniquement pas** être créé avec `pili-pos-android` v1.0.0. La garde n'est pas seulement une omission de l'UI : elle est appliquée côté serveur (Rust), dans `crates/pos-engine/src/commands`, avant tout appel au FDM.

- `commands/checkout.rs` : une vente sans ligne de produit est refusée (code `order_no_lines`). Une vente dont le total est négatif est refusée aussi (code `order_total_negative`) : un montant négatif ne passe que par un remboursement.
- `commands/tables.rs` : une addition provisoire sans ligne est refusée (`order_no_lines`). L'enregistrement d'une commande de table n'envoie rien quand aucune entrée n'est en attente : pas d'événement P vide.
- `commands/refund.rs` : un remboursement sans ligne à rembourser est refusé (`nothing_to_refund`).

Une vente dont le total vaut **0,00 EUR** (par exemple un article offert à prix nul) n'est pas un enregistrement zéro : elle porte ses lignes de produit. Elle est signée comme toute vente, en événement **N**, avec un ticket de caisse T.V.A. (FAQ producteurs Q206 et Q209 ; arrêté ministériel du 29 avril 2024, art. 5, 1°).

Tests : `checkout_of_an_empty_order_is_refused_without_calling_the_fdm`, `a_negative_sale_is_refused_bef` `faq_q206_a_zero_sale_is_signed_with_a_vat_ticket`.

**Déclaration formelle :** un enregistrement sans ligne de produit **n'est jamais possible** avec `pili-pos-android` v1.0.0, ni via l'UI opérateur, ni via la couche applicative (gardes serveur dans `checkout.rs`, `tables.rs` et `refund.rs`, cf. ci-dessus).

**Journal de commande (`order_entries`) :** chaque ajout, correction ou remboursement de ligne sur une commande ouverte est journalisé dans la table append-only `order_entries` (`crates/pos-db`, triggers SQL qui bloquent tout `DELETE`/`UPDATE` des champs journalisés), avec un `kind` parmi `entry` / `correction` / `refund` - conforme à l'exigence POS v2.2 §2 "toutes les données d'entrée, par ordre chronologique, y compris les corrections". `order_lines` reste la vue consolidée affichée/imprimée. Les entrées non encore transmises sont envoyées au FDM comme événement **P** (`sign_table_order`, "vente interrompue") lorsque l'opérateur quitte une table avec une commande non payée, ou dans son intégralité dans l'événement **N** final à l'encaissement ; une vente à emporter, vente directe, n'est jamais enregistrée en P (FAQ Q205 : `sign_table_order` la refuse) et se termine toujours par un N ; `sent_event_id` marque l'entrée comme transmise, en excluant tout double-comptage. Les lignes à quantité négative portent `negQuantityReason` (`crates/pos-fiscal`) : `CORRECTION` dans les événements P (commande / pré-addition - réduction avant encaissement) et `CORRECTION` ou `REFUND` dans l'événement N final selon qu'une vente d'origine est référencée ou non (`crates/pos-fiscal/src/event.rs`, `NegQuantityReason`). La correction d'une ligne avant paiement (quantité abaissée sur l'écran de commande) est rejouée sur la fixture UC230 (test `uc230_a_line_corrected_before_payment`). Dans les rapports X et Z, une correction de table est comptée sur l'événement P qui l'a transmise, même si la table est payée après le Z ; la vente finale qui la renvoie ne la recompte pas (FAQ Q303 ; `crates/pos-fiscal/src/report_totals.rs`).

**Déconnexion avec une table ouverte :** la déconnexion n'est pas bloquée. Quitter une commande de table, quel que soit l'écran visé, enregistre d'abord ses entrées en attente (événement P, vente interrompue : arrêté ministériel du 29 avril 2024, art. 6 ; FAQ Q205). Cette règle est appliquée par le moteur (`crates/pos-engine/src/commands/session.rs` : `do_leave_order`, `do_set_active_order`, `do_lock_session`) ;

l'interface se contente de les appeler lorsque l'opérateur change d'écran ou de commande, et le verrouillage automatique du moteur suit le même chemin. Si le FDM refuse ou ne répond pas, l'opérateur reste sur la commande ; seul le verrouillage de l'écran passe, et les entrées restent en attente jusqu'au prochain enregistrement, à l'addition ou à la vente. Sans FDM, aucune opération fiscale n'est de toute façon possible. La table reste listée comme ouverte sur l'écran des tables.

**Bouton "Addition" (pré-addition, événement P) :** `sign_pre_bill` imprime la facture consolidée au format PRO FORMA ("addition provisoire", POS v2.2 §1.4). Comme une pré-addition ne peut ni ajouter ni modifier de lignes, la commande enregistre d'abord les entrées de journal en attente via `sign_table_order` (même flux que ci-dessus), puis signe et imprime la pré-addition à partir des lignes consolidées.

## §III.E Enregistrement de la présence

**Description de l'implémentation :**

L'enregistrement du temps de présence est implémenté via l'event **S** (Work In / Work Out) du SCE 2.0. À la connexion d'un opérateur sur le POS, après saisie INSZ + PIN (ou présentation du badge, pour un Serveur ou un Apprenti, voir « Pointage par badge » ci-dessous), le moteur met le pointage d'entrée en file (`commands::auth::do_login`) et un processus de présence du moteur (`crates/pos-engine/src/background.rs`) appelle `commands::social::do_work_in`, sans que la connexion attende le FDM ; l'interface ne fait qu'afficher l'événement `presence-failed` si l'entrée n'est pas enregistrée. `do_work_in` :

1. Construit un `WorkInput` avec `pos_id`, `pos_date_time`, `terminal_id`, `employee_insz`, `device_id`.
2. Envoie un `signWorkIn` au FDM.
3. Sur succès, archive l'event S dans `fiscal_events` et, dans la même transaction, ouvre la prestation de l'employé dans `staff_shifts` (index local des prestations ouvertes, migration 9), sauf pour un événement d'entraînement : un S étiqueté T n'enregistre aucune présence et ne touche jamais `staff_shifts`. La signature est factorisée dans `commands::social::sign_work_event`, partagée par la connexion, la fin de service et l'enregistrement par le Gérant décrit ci-dessous.

Le même mécanisme côté `work_out`, mais à la fin de service seulement : le bouton « Verrouiller » de l'en-tête appelle `commands::session::do_lock_session` du moteur, qui enregistre la commande en cours puis verrouille la session, et l'employé reste en service (aucun S sortant ; à la reconnexion, aucun second `work_in` n'est émis tant que l'employé est en service). Le départ (`signWorkOut`) est signé depuis l'écran de connexion : l'employé en service touche sa tuile, puis « Fin de service » (avec son PIN s'il en a un) ; le moteur (`commands::social::do_end_shift`) contrôle le PIN, signe le départ au nom de l'employé, puis referme la session ; l'interface (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/feature/login/LoginScreen.kt`) ne fait que l'appeler.

**Enregistrement séparé pour les utilisateurs qui n'utilisent pas le POS :**

Le manuel de certification (§II.E) impose que la présence reste enregistrable pour le personnel qui n'utilise pas la caisse (cuisine, plonge). Deux fonctions du moteur le permettent, réservées au rôle Gérant (`Access::Admin`) :

- `commands::social::do_clock_in_employee` signe l'arrivée (`signWorkIn`) d'un autre employé ;
- `commands::social::do_clock_out_employee` signe son départ (`signWorkOut`).

Dans les deux cas l'événement S est celui de l'employé : le champ `employeeId` de la requête et la colonne `fiscal_events.user_insz` portent l'INSZ de l'employé, comme dans les cas d'usage M140 et M141. Le Gérant qui a fait l'enregistrement est conservé à part, dans `staff_shifts.started_by` (arrivée) et `staff_shifts.ended_by` (départ), pour l'audit ; il n'apparaît pas dans la requête envoyée au FDM. Depuis l'écran « Config » de la caisse, le panneau « Employés » affiche pour chaque autre employé actif « En service depuis HH:MM » ou « Hors service » et un seul bouton, « Arrivée » ou « Départ » (le Gérant n'a pas de bouton sur sa propre ligne : sa présence passe par la connexion et « Fin de service »). L'interface (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/feature/settings/SettingsScreen.kt`) ne fait qu'appeler ces deux commandes, puis relire la liste des employés et des prestations ouvertes.

Règles du moteur, dans cet ordre, sans rien signer ni faire avancer de compteur en cas de refus : appelant qui n'est pas Gérant (`forbidden`) ; mode formation actif (`presence_training`) ; utilisateur système (`system_user`) ; employé inconnu ou désactivé (`user_unknown`) ; le Gérant lui-même (`presence_self`) ; arrivée alors que l'employé est déjà en service (`presence_already_in`) ; départ alors qu'il ne l'est pas (`presence_not_in`). Le mode formation est refusé parce qu'un événement T n'enregistre aucune présence : enregistrer celle d'un autre employé en formation serait trompeur. Si le FDM est injoignable ou refuse, la commande répond « échec » avec le texte du FDM, rien n'est archivé et `staff_shifts` ne change pas ; l'interface désactive en outre les deux boutons avec la mention « FDM indisponible : pointage impossible » tant que le FDM n'est pas connecté. La désactivation d'un employé en service est refusée (`deactivate_on_shift`) : le Gérant enregistre d'abord son départ.

Les employés qui ne se servent jamais de la caisse sont créés avec le profil « Présence seule » (`users.presence_only = 1`, rôle `waiter` en base, aucun droit). Un tel employé n'a pas de PIN, n'apparaît pas sur les tuiles de l'écran de connexion et ne peut ni se connecter, ni choisir un PIN, ni terminer son service lui-même (erreur `presence_only_user`) : seul le Gérant enregistre son arrivée et son départ, ou son propre badge s'il en a reçu un (« Pointage par badge » ci-dessous). Ses événements S figurent dans les rapports par utilisateur comme ceux de tout employé.

**Pointage par badge (lecteur de badges) :**

Le manuel de certification (§II.E) et la description du POS (v2.2 §1.9) acceptent « n'importe quelle routine d'inscription » à condition que la caisse crée les données correctes pour l'event S ; ils ne prescrivent pas comment l'utilisateur s'identifie auprès du POS (v2.2 §6.4 : le POS relie son identification interne au NISS, qui n'est jamais imprimé). pili-pos propose donc en option un badge (carte ou porte-clés) comme identification interne : le Gérant lie un badge à un employé de la table `users`, lequel porte déjà le NISS. Le badge ne remplace jamais le NISS : l'event S reste signé par le FDM avec le NISS de l'employé. Le réglage « Lecteur de badges » est désactivé par défaut ; désactivé, aucune lecture n'a lieu et toute commande de badge est refusée (`badge_reader_off`). Le lecteur est un lecteur USB qui se comporte comme un clavier (il tape le numéro du badge puis Entrée) ; sur le matériel Android qui déclare un lecteur NFC (`PackageManager.FEATURE_NFC`), le réglage propose aussi « NFC intégré ». La tablette Sunmi D3 Pro n'a pas de NFC dans son unité principale : un lecteur USB y est branché.

- **Passage d'un badge :** `commands::badges::do_badge_tap` normalise le numéro lu, le transforme en empreinte (HMAC-SHA256 avec une clé propre au terminal, voir ci-dessous), retrouve l'employé lié et décide, dans cet ordre : badge inconnu (`badge_unknown`) ou retiré (`badge_revoked`), refus journalisés ; session ouverte (n'importe quel écran) : bascule de présence de l'employé du badge, **sans déconnecter l'opérateur** (le badge de l'opérateur lui-même est refusé, `presence_self` : sa présence passe par la connexion et « Fin de service ») ; écran de connexion en mode « Fin de service » : départ seul, sans ouvrir de session ; écran de connexion avec la connexion par badge activée : ouverture de session comme une connexion par PIN, pour un Serveur ou un Apprenti seulement ; sinon bascule de présence (personnel « Présence seule », ou connexion par badge désactivée).
- **Event S :** la bascule appelle `commands::social::sign_work_event` avec l'employé du badge : `signWorkIn` s'il n'est pas en service, `signWorkOut` s'il l'est (index local `staff_shifts`). La requête est celle des cas d'usage M140 et M141 pour cet employé : `employeeId` et `fiscal_events.user_insz` portent son INSZ, `staff_shifts.started_by` et `ended_by` restent vides (enregistrement par l'employé lui-même). Le badge n'apparaît que dans le journal `badge_scans`, jamais dans l'event.
- **FDM indisponible :** le passage signe ; si le FDM est injoignable ou refuse, le résultat est « échec de présence » avec le texte du FDM, rien n'est archivé, `staff_shifts` ne change pas, et l'interface affiche ce texte. Aucune mise en tampon : un event S n'est jamais signé en différé. La connexion par badge fonctionne sans FDM, exactement comme la connexion par PIN (le processus de présence signale l'échec par l'événement `presence-failed`).
- **Mode formation :** la bascule de présence est refusée (`presence_training`), pour la même raison que l'enregistrement par le Gérant : un event T n'enregistre aucune présence. La connexion par badge reste possible (le processus de présence envoie alors un `signWorkIn` d'entraînement, qui n'ouvre aucune prestation), de même que le lien et le retrait d'un badge (aucun event fiscal).
- **Double passage :** le moteur ignore un second passage du même badge tant que le premier est en cours de traitement et pendant 10 secondes après sa fin, quel qu'en soit le résultat (refus compris). Un badge

oublié sur le lecteur ou un double geste ne peut donc ni pointer l'arrivée puis le départ, ni remplir le journal.

- **Les droits de Gérant exigent toujours le PIN :** le badge d'un Gérant sur l'écran de connexion se borne à sélectionner ce Gérant et à ouvrir son clavier de PIN (`pin_required`) ; la session est ouverte par la connexion habituelle (INSZ + PIN, avec son verrouillage). Aucune action réservée au Gérant n'est accessible par un badge seul.
- **Lier, retirer :** `commands::badges::do_link_badge` (Gérant, avec son propre PIN, puisqu'un badge est un moyen de connexion) et `commands::badges::do_revoke_badge` (Gérant, sans PIN : retirer ne fait que supprimer un accès). Un employé a au plus un badge actif et un badge actif n'appartient qu'à un employé ; la désactivation d'un employé retire son badge dans la même transaction. Chaque lien, retrait et changement de réglage est journalisé (`badge_log`, qui, quand).
- **Données :** seule une empreinte HMAC-SHA256 du numéro du badge est conservée, avec une clé de 32 octets tirée au hasard et stockée dans un fichier (`badge.key`) hors de `pos.db` : elle n'est donc ni dans les sauvegardes ni dans l'export nocturne, et elle ne quitte jamais le terminal. Le numéro lui-même n'est écrit nulle part (ni base, ni journaux, ni event). Voir *SECURITE_BD_FR.md* et *PROFIL_ENTREPRISE_FR.md* §6 pour le RGPD.

Aucune nouvelle catégorie d'event n'est créée : le badge n'est qu'un déclencheur supplémentaire, équivalent aux boutons « Arrivée » et « Départ » de l'enregistrement par le Gérant et à la connexion par PIN.

| Item | Réponse |
|---|---|
| ☒ Description de l'utilisation | *Voir : MANUEL_UTILISATEUR_FR.md* §3 "Pointage d'arrivée et de départ", §12.3 "Employés" et §12.13 "Lecteur de badges" |
| ☒ L'enregistrement du temps de présence est lié aux boutons "Work In / Work Out" | Oui (le passage d'un badge est un déclencheur équivalent, voir ci-dessus) |

## §III.F Fonctionnalités avancées

| Fonction | Utilisée | UseCase | Statut |
|---|---|---|---|
| Normal (N) | Obligatoire (☒) | M110 | OK |
| Remboursement total | ☒ | M111 | OK *Voir : FLUX_REMBOURSEMENT_FR.md* §1 |
| Remboursement partiel | ☒ (déclaré **M112B** - ticket séparé) | M112 | OK *Voir : FLUX_REMBOURSEMENT_FR.md* §1 |
| Report (R) | Obligatoire (☒) | Reports | OK (M180-M184) |
| Social (S) | Obligatoire (☒) | M140/141 | OK |
| Pro Forma (ticket non-TVA) | ☒ | M121 | OK |
| Commande en attente | ☒ | - | OK (gestion d'ordre ouvert sur table) |
| Prebill Pro Forma | ☒ | M123 | OK |
| Tous les events P sont imprimables | ☒ Format papier | - | OK |
| Gestion des cost centers (attribution / division / fusion) | ☐ | M122 / UC240-243 / UC250-252 | Hors périmètre MVP (Advanced Set). Fixtures parsent mais UI non implémentée. |
| Attribution | ☐ | - | Idem |
| Division | ☐ | UC240/241/243 | Idem |
| Fusion | ☐ | UC250/251/252 | Idem |

| Fonction | Utilisée | UseCase | Statut |
|---|---|---|---|
| Gestion d'addition (division table / addition / parts égales) | ☐ | - | Idem (Advanced Set) |
| Division de table | ☐ | - | Idem |
| Division d'addition (parts égales) | ☐ | - | Idem |
| Division d'addition (montants égaux) | ☐ | - | Idem |
| Invoice (I) | ☐ | M150 | Non disponible en v1.0.0. Le code (`crates/pos-peppol`) existe mais le feature flag `peppol` n'est pas activé dans la build cert : `pos-engine` est construit avec la seule fonction `cert` (`android/rust/pos-ffi/Cargo.toml` ; `cert = ["hubrise"]` dans `crates/pos-engine/Cargo.toml` : HubRise est dans l'APK, seul `peppol` en est absent), comme sur Linux. Mise à jour certifiée ultérieure *Voir : DECISION_PEPPOL_AP_FR.md* |
| Note de crédit | ☐ | - | Reporté avec Peppol / event I (cf. *FLUX_REMBOURSEMENT_FR.md* §3) |
| Copy (C) | ☒ | M160 | OK |
| Training (T) | ☒ | M171 | OK |
| Gestion des cash flows | ☒ | M130 | OK |
| Gestion du tiroir-caisse | ☒ | M131 | OK |
| **Moyens de paiement** | | | |
| Cash | ☒ | - | OK (arrondi belge implémenté) |
| Carte de débit | ☒ | - | OK |
| Carte de crédit | ☒ | - | OK (même flux que débit) |
| Crédit client | ☐ | - | Pas de système clients (B2C) |
| Facturation à l'hôtel | ☐ | - | Non supporté (POS destiné à la restauration) |
| Chèque repas | ☒ | ROUND4, ROUND5 | Chèques-repas électroniques (`CHEQUE_MEAL`, id "3"), montant exact jamais arrondi, sans rendu, tiroir fermé |

| Fonction | Utilisée | UseCase | Statut |
|---|---|---|---|
| Paiement en plusieurs moyens (un ticket) | ☒ | UC4119, ROUND4, ROUND5 | Carte et chèques-repas pour un montant exact, espèces pour le reste (seul le reste en espèces est arrondi, ligne ROUNDING à part). Une ligne PAYMENT par moyen (`crates/pos-fiscal/src/payment.rs`) |
| Vente sans paiement (client parti sans payer) | ☒ | UC4109 | Gérant + PIN : N signé avec `financials: []`, ticket TVA “Non paye”, ni tiroir ni paiement, jamais remboursable (`crates/pos-engine/src/commands/unpaid.rs`). La description POS v2.2 ne prévoit des `financials` vides que pour un montant nul ; pili-pos-android suit le cas d’utilisation officiel UC4109_2 (58 EUR, `financials: []`) |
| Changement de paiement (M132) | ☐ | M132 | Reporté (cf. *FLUX_REMBOURSEMENT_FR.md* §2) |
| Vouchers à but multiple | ☐ | UC260/261 | Hors MVP |
| Vouchers à but unique | ☐ | UC270/271 | Hors MVP |
| Devise étrangère | ☐ | UC296 | Déclaré : EUR uniquement |
| Arrondissement des paiements cash | ☒ | Roundings | Obligatoire pour le cash, implémenté (arrondi belge à 5 centimes) |
| Arrondissement carte EFT | ☐ | Roundings | Optionnel - pas d’arrondi sur paiements EFT |
| Gestion des livraisons | ☒ | - | Commandes en ligne uniquement (plateformes de livraison et webshop, par HubRise, activé par installation) : livraison ou enlèvement, signées par le FDM (N, ou P puis N à la remise). Aucune livraison n’est saisie à la caisse. *Voir : COMMANDES_EN_LIGNE_FR.md* |

| Fonction | Utilisée | UseCase | Statut |
|---|---|---|---|
| Réductions | ☒ | UC223 | Uniquement les remises des commandes en ligne (code promo, réduction du webshop ou de la plateforme) : `priceChanges` de portée `EVENT`, type `PUBLIC`, réparties sur les lignes de produits (`crates/pos-engine/src/commands/price_chang` `crates/pos-fiscal/src/price_change.rs`, migration 23, test `crates/pos-engine/tests/hubrise_discounts.` Aucune remise n'est saisie à la caisse : les modificateurs +/- sur prix oui, les réductions globales non. *Voir : COMMAN-DES_EN_LIGNE_FR.md* §5 |
| Produits composés | ☐ | UC280 | Fixtures parsent, UI non implémentée (Advanced Set) |
| Modification de prix ligne | ☐ | UC220 | Pas de modification de prix de portée ligne (ni à la caisse ni dans les commandes en ligne) |
| Modification de prix event | ☒ | UC223 | Commandes en ligne uniquement (cf. « Réductions ») : remise de portée `EVENT` répartie sur les lignes. La transaction construite par pili-pos-android est comparée à la fixture officielle UC223 (test `crates/pos-fiscal/tests/price_change_uc.rs` Pas de modification de prix d'événement à la caisse |

| Fonction | Utilisée | UseCase | Statut |
|---|---|---|---|
| Modification de prix multiple | ☐ | UC224 | Plusieurs remises de portée `EVENT` sont possibles sur une commande en ligne (un `groupingId` par remise), mais le scénario UC224 (remise de ligne combinée à une remise de ticket) n'est pas rejoué : pas de fixture UC224 dans les tests |
| Corrections (modif quantité) | ☒ | UC230 | Correction d'une ligne avant paiement : quantité abaissée ou ligne retirée sur une commande ouverte, journalisée (`order_entries`, `kind = correction`) et envoyée avec `negQuantityReason CORRECTION` ; fixture UC230 rejouée (test `uc230_a_line_corrected_before_p` Après encaissement, la correction est un remboursement (M111/M112B) |
| Article offert/gratuit (100% réduction) | ☐ | - | Hors MVP |
| Happy Hour programmé | ☐ | - | Hors MVP |
| Gestion des clients | ☐ | - | B2C, pas de base clients |
| Réduction membre | ☐ | - | Idem |
| Mensualités | ☐ | - | Idem |
| Prix libre / "prix du jour" | ☒ | - | Implémenté : colonne `menu_items.open_price` ; le produit `22010` (« Dish of the day » dans le manuel et le seed, affiché « Plat du jour » en français) du seed de certification est marqué `open_price = 1`, le prix est saisi par l'opérateur à l'ajout au lieu du prix catalogue. |
| Prix au poids | ☐ | - | Pas de balance |
| Ticket de service (bar / cuisine) | ☒ | - | Imprimante cuisine dédiée, ticket non-fiscal sans prix |
| Autres | ☐ | - | - |

**Remarque importante - facturation B2B Peppol**

À partir du 01.01.2026, toutes les factures B2B doivent être émises via le réseau Peppol (BIS Billing 3.0). Cette obligation est **hors périmètre de la build cert v1.0.0** de `pili-pos-android` :

- **Sérialiseur UBL** et trait `AccessPoint` existent dans le workspace (crate `pos-peppol`, deux implémentations : `LoggingAccessPoint` no-op et `StorecoveAccessPoint` squelette de production) et sont couverts par des tests unitaires, mais ce code n'est **pas compilé dans la build cert** : la bibliothèque `libpos_ffi.so` est construite avec la seule fonction `cert` du moteur, qui n'active pas `peppol` mais active `hubrise` (`android/rust/pos-ffi/Cargo.toml` ; `cert = ["hubrise"]` dans `crates/pos-engine/Cargo.toml`). L'événement fiscal **I (Invoice)** n'est donc pas disponible en v1.0.0.
- **Table d'audit** `peppol_deliveries` existe dans le schéma pour une activation future, mais n'est alimentée par aucun flux en v1.0.0.
- **Mise à jour certifiée ultérieure** : câblage de l'Access Point (Storecove pressenti - voir *DECISION_PEPPOL_AP_FR.md*) et activation du feature flag `peppol`, suivis d'une nouvelle demande de certification (procédure §II.E).

**En attendant cette mise à jour**, les factures B2B du restaurant (cas exceptionnels - client entreprise, établissement voisin) sont émises **en dehors de `pili-pos-android`**, via l'outil de comptabilité/facturation du restaurant. `pili-pos-android` reste la seule source pour les events N/P/T/C/F/S/R fiscalisés ; aucune facture B2B n'est émise ni imprimée par le POS en v1.0.0.

## §III.G CostCenterChange (Gestion de Table)

| Item | Réponse |
|---|---|
| Le POS permet le changement de cost center vers ROOM (chambre) | ☐ Non (POS destiné à la restauration, pas d'hôtellerie) |
| Le POS permet de recalculer le changement de prix en cas de changement de cost center | ☐ Non (changement de cost center pas implémenté en MVP) |
| Les liens dans la base de données sont décrits | N/A (fonctionnalité non implémentée) |

## §III.H Emission des tickets numériques

L'émission de tickets sous forme numérique est implémentée par **e-mail** (Scaleway Transactional Email), en complément du **QR Code Type 2 imprimé sur chaque ticket papier** qui pointe vers l'URL de vérification publique du FDM SPF Finances.

- **Choix opérateur avant signature.** Avant l'encaissement, la copie ou le remboursement, l'opérateur choisit Papier / E-mail / Les deux. Le support retenu fait partie du corps **signé** par le FDM (`ticketMedium`, POS spec v2.2 §1) : il n'est jamais changé après coup. Une adresse invalide, vide, ou une demande d'e-mail sur un terminal non configuré est refusée **avant** tout appel au FDM.
- **Émission numérique :** l'e-mail contient le texte du ticket, identique au caractère près à ce qui s'imprime sur papier, et - pour les ventes/remboursements (événements N) uniquement, le FDM ne renvoyant l'URL de vérification que pour ceux-là - le lien de vérification SPF Finances ainsi qu'un QR Code en pièce jointe. Le QR Code imprimé (25×25 impossible à faire tenir l'URL, donc version 3, 29×29 modules) reste présent sur **tout** ticket fiscal, quel que soit le support choisi.
- **Garantie de remise :** la mise en file d'attente de l'e-mail se fait dans la **même transaction SQLite** que l'archivage de l'événement fiscal signé - un ticket numérique promis ne peut pas se perdre silencieusement. L'envoi est tenté immédiatement (budget 8 s) ; s'il n'aboutit pas dans ce délai pour un ticket "E-mail seul", le POS imprime automatiquement l'original en secours - le client repart toujours avec un ticket. Pour "Les deux", le papier est de toute façon toujours imprimé, l'e-mail suit en meilleur effort avec des réessais en tâche de fond (recul exponentiel, jusqu'à 5 tentatives sur ~1h30).
- **Impression unique de l'original (§8.1) :** un ticket original n'est imprimé qu'une seule fois, contrôlé au niveau base de données (`original_prints`) ; toute reproduction ultérieure passe par l'événement C (copie), jamais par une seconde impression silencieuse de l'original - y compris pour rattraper une impression papier manquée. Ce rattrapage (`print_missing_original`) ne concerne qu'un original jamais

imprimé : le ticket est reconstruit à partir de l'archive, pour une vente (N) ou une copie (C) seulement (code `print_original_unsupported` sinon), et il est refusé si l'original est déjà sorti ou déjà envoyé par e-mail.

- **RGPD :** l'adresse e-mail en clair n'est jamais conservée plus de 72 h, et est effacée immédiatement dès l'envoi réussi ou le repli papier. Seul un indice masqué (ex. `j***@e***.be`) subsiste pour la lisibilité du support ; l'adresse n'apparaît jamais dans les journaux ni dans l'écran "Tickets numériques" des Paramètres.
- **Conservation du QR :** l'URL de vérification reste valide pendant les 10 ans de conservation légale (Article 60 §4 Code TVA).

*Voir : TICKET_NUMERIQUE_FR.md* pour le détail complet de l'implémentation, la table exigence → code → test, et `MANUEL_INSTALLATION_FR.md` §4.2 pour la configuration (`pos.json::digital_ticket`).

## §III.I Terminaux et appareils

| Item | Réponse |
|---|---|
| Le POS permet-il de travailler avec plusieurs périphériques sur un terminal (roaming terminals) ? | ☐ Non (configuration mono-terminal, mono-imprimante, mono-tiroir) |

## §III.J Périodes

| Item | Réponse |
|---|---|
| Le POS permet-il de travailler à la fois avec des périodes d'ouverture (bookingDate) et des créneaux horaires (bookingPeriod) ? | ☒ Oui |
| Configuration | `bookingDate` (date de service fiscale) avance automatiquement au signReportTurnoverZ (clôture journalière). `bookingPeriodId` est **persisté** en base (table `booking_periods`) - une période court du premier événement suivant un Z jusqu'au Z suivant, et porte la date d'ouverture (fuseau Europe/Brussels). Un seul redémarrage du POS un jour donné ne coupe donc jamais la journée en deux périodes : au démarrage, le POS reprend la période ouverte si elle existe, sinon en ouvre une nouvelle. Des triggers SQL interdisent la réouverture et la suppression d'une période close ; un index unique garantit qu'une seule période est ouverte à la fois. Une clôture Z **en mode formation** ne clôture pas la période (ni ne consomme de numéro de Z, cf. §III.L). Aucune configuration manuelle requise par l'opérateur. *Voir : DOCUMENT_PROCESSUS_TECHNIQUE_FR.md* §4 et *FLUX_DONNEES_FR.md* §5. |

## §III.K Tiroirs-Caisse

| Item | Réponse |
|---|---|
| Est-ce que cette fonctionnalité est disponible sur votre POS ?<br>**Si oui uniquement :** | ☒ Oui |

| Item | Réponse |
|---|---|
| S'agit-il d'un tiroir-caisse / portefeuille physique ? | ☒ Oui (tiroir RJ11 raccordé à l'imprimante) |
| Est-il possible d'avoir plusieurs tiroirs par POS ou par utilisateur ? | ☐ Non (1 seul tiroir physique par POS) |
| S'agit-il d'un tiroir-caisse virtuel ? | ☐ Non |
| Aucun (tiroir-caisse par défaut "hardcodé" pour les mutations GraphQL) | ☐ Non |
| Implémentation | Le tiroir est ouvert par séquence ESC/POS `0x1B 0x70 0x00 0x19 0xFA` envoyée à l'imprimante au moment d'un paiement cash (close de vente) ou via la commande manuelle `commands::financial::do_open_drawer_fiscal` (event F-WAR émis et archivé). Les rapports utilisateur (M183) ventilent les paiements par mode (cash / card / autre). |

## §III.L Rapports

| Item | Réponse |
|---|---|
| Rapports Z : Un seul rapport par BookingDate | ☒ Oui (la clôture Z incrémente `bookingDate` ; deux Z dans la même journée fiscale sont refusés au niveau applicatif) |
| Rapports Financier Z et Utilisateurs Z : sont-ils liés et émis ensemble ? | ☒ Oui (M184 - mutation GraphQL aliasant `signReportTurnoverZ` + `signReportUserZ` dans une seule opération atomique - cf. *COUVERTURE_RAPPORTS_FR.md* §M184) |
| Rapports Financier Z et Utilisateurs Z : sont-ils émis automatiquement ? | ☒ Oui, en filet de sécurité. Le Z normal est celui du Gérant (« Fin de journée », après recompte de caisse). Si une journée reste ouverte sans Z, la caisse la clôture seule (`commands/report.rs::close_overdue_day`, au démarrage puis chaque minute, une fois la journée passée selon `day_end_hour`) avec le même Z combiné, signé au nom de l'identité système `00000000029` (FAQ Q501c, test `crates/pos-engine/tests/spf_e2e_auto_z.rs`). Tant que ce Z n'est pas fait, aucun nouvel événement n'entre dans l'ancienne journée. Pas de Z automatique en mode formation ni pour une journée sans événement ; si le FDM est injoignable, le Z est retenté à la minute suivante. |
| Le POS dispose-t-il d'autres rapports que les rapports obligatoires X et Z ? | ☐ Non. Seuls X (turnover + user) et Z (turnover + user) sont implémentés. |

| Item | Réponse |
|---|---|
| Calcul des totaux (turnover, users, posDevices, fdmDevices) | Calculés par `crates/pos-fiscal/src/report_totals.rs` **exclusivement à partir des événements fiscaux archivés et signés** de la période d'ouverture en cours (`booking_periods`, cf. §III.J), jamais des tables `orders`/`order_lines`/`payments` mutables - conforme POS v2.2 §2.2.2. Détail : `departments`, `payments` et `negQuantities` sont calculés à partir des événements N uniquement ; `vats` est la somme du `vatCalc` renvoyé par le FDM pour chaque N (repli sur les taux standard A/B/C si absent) ; `users[].totalAmount` est la somme des `transactionTotal` des N de l'employé, et la plage d'activité (`first`/`lastPosDateTime`) couvre tous ses événements **sauf** les rapports R eux-mêmes. Chaque moyen de paiement a son propre `financials.id` (spec §2.2.2) - `"1"` espèces, `"2"` carte - pour que les rapports les distinguent. Tous les montants sont additionnés en centimes (pas de flottants) pour éviter la perte de centime observée avant ce correctif (3,04 € devenait 3,03 €). |
| Rapport Z en mode formation | ☐ Un Z de formation **ne consomme pas de numéro de Z** et **ne clôture pas** la période d'ouverture (`booking_periods` reste ouverte) - cohérent avec le principe que les événements T ne comptent jamais fiscalement. |
| Rejeu du scénario officiel Reports v1.3 | Les 8 tickets T1-T8 du scénario SPF (`docs/20260821_UC_Reports_1.3/`) sont rejoués deux fois par les tests : purement via `report_totals` (`crates/pos-fiscal/tests/report_totals_fixtures.rs`) et de bout en bout via les commandes réelles (`crates/pos-engine/tests/spf_e2e_report_fixtures.rs`). Les deux correspondent aux rapports publiés M180-M184, avec un écart assumé et documenté dans le test : le rapport publié compte 3 tickets P pour 198 € (T1 exclu), alors que le scénario enregistre 4 événements P (commandes T1/T5/T6 + pré-addition T7) pour 250 € ; la spec définit `ticketCount` comme le nombre d'événements du label sur la `bookingDate` (§2.2.2 turnover), donc pili-pos-android rapporte 4 et 250 € - conforme à la spec, différent du chiffre publié dans la fixture. Les paiements carte sont toujours `CARD_DEBIT` (l'opérateur ne distingue pas débit/crédit à l'encaissement). |

## §III.M Intégration des tiers-parties

| Nom | Statut |
|---|---|
| Deliveroo | ☒ Oui, par HubRise (activé par installation) |
| TakeAway.com | ☒ Oui, par HubRise (activé par installation) |
| Uber Eats | ☒ Oui, par HubRise (activé par installation) |
| Autres partenaires | ☒ Webshop de l'établissement, par HubRise. Chaque commande se termine par un N signé sur ce terminal ; détail, interrupteur par installation et journal : *COMMANDES_EN_LIGNE_FR.md* |

**Remarque :** Scaleway (Transactional Email, §III.H) n'est pas une intégration de plateforme de commande - c'est un sous-traitant technique utilisé uniquement pour l'envoi du ticket numérique par e-mail, à la demande du client, jamais pour recevoir des commandes. *Voir : PROFIL_ENTREPRISE_FR.md* §6 pour le détail RGPD (base légale, durée de conservation, localisation).

**Données personnelles des commandes en ligne :** HubRise est la source des commandes en ligne et leur concentrateur technique (sous-traitant au sens du RGPD pour le transit des commandes). Il transmet à la caisse les données du client de la commande (nom, téléphone, adresse de livraison, selon la commande). La caisse les conserve avec la commande (`orders.raw_hubrise_json` et `hubrise_events.payload_json`, JSON reçu de HubRise, sans purge automatique) et les imprime sur le bon client non fiscal remis au livreur ou au client (`crates/pos-print/src/customer_slip.rs`). Rien de cela n'existe tant que `hubrise_enabled` est coupé (valeur par défaut, donc machine de test). Détail : *COMMANDES_EN_LIGNE_FR.md* §3 bis ; RGPD : *PROFIL_ENTREPRISE_FR.md* §6.

**Remarque :** le point de réception de l'export nocturne (section `export` de `pos.json`, facultative) n'est pas une intégration de plateforme de commande : il ne reçoit que des fichiers que la caisse lui envoie, jamais de commande. *Voir : EXPORT_NOCTURNE_FR.md*.

#### Remarques sur les fonctionnalités du POS

Toutes les fonctionnalités déclarées ☒ sont couvertes par des tests automatisés (unit, intégration, e2e applicatif sur les commandes du moteur). Voir *RAPPORT_TESTS_BASELINE_FR.md* pour le détail des tests par binaire, régénéré sur la révision soumise par `scripts/generate_test_report.py`.

### §III - Lignes de produit prédéfinies

**Statut :** la machine de test est initialisée avec exactement les 25 produits du manuel (Dry Martini productId 10006, Burger of the Chef 22001, Carpaccio Beef 20001, Catch of the day 22005, Dame Blanche 25003, Crème brûlée 25008, etc.), leurs départements, prix et codes TVA, depuis `src-tauri/seed_spf_test.sql`, fourni comme `seed.sql` du lot de provisionnement de la machine de test. *Voir : LIGNES_PRODUITS_FR.md*.

Ce seed sera chargé sur la machine de test à l'Intake. Les UC du Bloc IV (procédure GraphQL) référencent ces productId précis ; le seed SPF garantit que les UC peuvent être exécutés à l'identique.

---

## §IV - PROCÉDURE DE COMMUNICATION GRAPHQL

Le manuel comprend une procédure de test détaillée par mutation (M110, M111, M112, M121, M122, M123, M130, M131, M132, M140/141, M150, M160, M171, M180-184) plus les Advanced UC. Pour chaque mutation, le manuel attend : 1. Un tableau "Champs à vérifier" coché. 2. Une grille TVA vérifiée. 3. La capture JSON request + response. 4. Le ticket imprimé.

**Statut pili-pos-android :**

- Les 22 UC du Basic Set v1.8, les 5 UC de Reports v1.3 et les 141 fixtures (59 UC) de l'Advanced Set v1.6 sont chargées et parsées par les tests automatisés (*Voir : COUVERTURE_FIXTURES_FR.md, COUVERTURE_RAPPORTS_FR.md, COUVERTURE_ADVANCED_SET_FR.md*).
- Dans le Basic Set, 18 des 22 UC ont un test automatisé ; les 4 autres sont exclues et déclarées : M150 (event I), M122 et M132, hors périmètre ; M112A, non applicable (le remboursement partiel est toujours émis en ticket séparé, M112B). Dans Reports, M180 à M184 ont des tests de fixtures (M181 et M183 par le Z combiné M184) et les 8 tickets T1 à T8 sont rejoués. Les tests e2e applicatifs (commande du moteur → tx SQLite → archive) rejouent ces UC avec la requête construite par le moteur. Sur Android, la suite instrumentée rejoue les mêmes UC sur l'émulateur par l'appel FFI de même nom (`docs/android/FFI.md` §11) ; les JSON de 17 UC (14 Basic Set, 3 Reports) : À effectuer (EV-06 : résultats UC sur l'émulateur Android). *Voir : RAPPORT_TESTS_BASELINE_FR.md*.
- **Document détaillé par UC** avec JSON request/response et ticket : *Voir : RESULTATS_TESTS_UC_FR.pdf*.

Récap mappings UC <-> commande du moteur <-> test :

| UC | Mutation | Commande | Test |
|---|---|---|---|
| M110 | signSale | `checkout::checkout` | `spf_e2e_basic::m110_*` |
| M111 | signSale refund full | `refund::refund_checkout` | `spf_e2e_basic::m111_*` |
| M112B | signSale refund partial (événement séparé, chaîné) | `refund::refund_checkout` (multi-step) | `spf_e2e_basic::m112b_*` |
| M121 | signOrder | `tables::sign_table_order` | `spf_e2e_basic::m121_*` |
| M123 | signPreBill | `tables::sign_pre_bill` | `spf_e2e_basic::m123_*` |
| M130 | signMoneyInOut | `financial::money_in_out` | `spf_e2e_basic::m130_*` |
| M131 | signDrawerOpen | `financial::open_drawer` | `spf_e2e_basic::m131_*` |
| M140 | signWorkIn | `social::work_in` | `spf_e2e_basic::m140_*` |
| M141 | signWorkOut | `social::work_out` | `spf_e2e_basic::m141_*` |
| M160 | signCopy | `copy::copy_ticket` | `spf_e2e_basic::m160_*` |
| M170/171/172 | signSale/Copy training | (mêmes commandes, flag is_training) | `spf_e2e_basic::m170_*` / `m171_*` / `m172_*` |
| M150 | signInvoice | `invoice::sign_invoice_fiscal` | `spf_e2e_basic::m150_*` |
| M161 | signCopy (commande P) | `copy::copy_ticket` | `spf_e2e_basic::m161_*` |
| M180 | signReportTurnoverX | `report::generate_x_report` | `spf_e2e_reports::m180_*` |
| M182 | signReportUserX | `report::generate_user_x_report` | `spf_e2e_reports::m182_*` |
| M184 (Z combiné) | signReportTurnoverZ + signReportUserZ (un document) | `report::generate_z_report` | `spf_e2e_reports::m184_*` |

La ligne M150 ci-dessus décrit la couverture de test (code + e2e réels, sous les features par défaut) ; elle n'implique pas que l'événement I soit disponible dans la build cert v1.0.0 soumise - voir §III.F et *DECISION_PEPPOL_AP_FR.md*.

## §IV - Arrondissement des méthodes de paiement

Le manuel demande les tests de base 1 et 2 dans tous les cas, le test de base 3 si le POS n'arrondit que les paiements en espèces, et le test de base 4 seulement si le POS arrondit tous les moyens de paiement. pili-pos-android n'arrondit **que les espèces**, au 0,05 EUR le plus proche, dans une ligne `ROUNDING` à part ; la carte et les chèques-repas sont des montants exacts, jamais arrondis, et seul le reste payé en espèces est arrondi (`crates/pos-fiscal/src/payment.rs`). Les tests de base 1, 2 et 3 s'appliquent donc ; le test de base 4 (multi-paiements 2, POS qui arrondit tous les moyens de paiement) ne s'applique pas.

| Test du manuel | Situation | Scénario | À payer / arrondi attendus | Test automatisé | Preuve JSON |
|---|---|---|---|---|---|
| Base 1 : arrondi du cash | 1 | Dame Blanche 11,52 EUR (B), cash seul | 11,50 EUR / -0,02 EUR | `round1_11_52_is_paid_in_cash_11_50` | `evidence/ROUND1*` |
| Base 1 : arrondi du cash | 2 | Tapas Variation 3,34 EUR (B), cash seul | 3,35 EUR / +0,01 EUR | `round2_3_34_is_paid_in_cash_3_35` | `evidence/ROUND2*` |
| Base 1 : arrondi du cash | 3 | Dame Blanche + Tapas Variation, 14,86 EUR, cash seul | 14,85 EUR / -0,01 EUR | `round3_14_86_is_paid_in_cash_14_85` | `evidence/ROUND3*` |
| Base 2 : chèque-repas | 4 | Dame Blanche + Tapas Variation ; 11,52 EUR en chèques-repas, le solde en cash | 3,35 EUR en cash / +0,01 EUR | `round4_meal_vouchers_then_cash_ROUND` | `evidence/ROUND4*` |
| Base 3 : multi-paiements 1 | 5 | Dame Blanche + Tapas Variation ; 3,13 EUR chèques-repas, 3,97 EUR carte de débit, le solde avec 10 EUR en cash | 7,75 EUR en cash, rendu 2,25 EUR / -0,01 EUR | `round5_meal_vouchers_card_then_cash_ROUND` | `evidence/ROUND5*` |
| Base 4 : multi-paiements 2 | 5 | (POS qui arrondit tous les moyens de paiement) | ☐ Non applicable | - | - |

Un remboursement en espèces est arrondi de la même façon, comme dans l'exemple de ticket de remboursement du SPF (`docs/20260904_Example_tickets/`) : un Coca Cola de 3,04 EUR remboursé est rendu 3,05 EUR, ligne `PAYMENT` de -3,04 puis ligne `ROUNDING` de -0,01, "Arrondi -0,01" sur le ticket, 3,05 EUR sortis du tiroir (test `a_cash_refund_is_rounded_like_the_spf_example`, `crates/pos-engine/tests/spf_e2e_rounding.rs`).

Les tickets correspondants figurent dans *TICKETS_EXEMPLES_FR.md*.

---

# §V - COMPORTEMENT DU POS AUX RÉPONSES DU FDM

**Statut implémentation :** la table `FdmMessageTables` (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/i1` engendrée à partir des mêmes textes que le fichier `ui/src/utils/fdmMessages.ts` de la version Linux) mappe **23 codes FDM** (les 21 du tableau ci-dessous, plus `UPDATE_POS_VATNO` et `UPDATE_POS_ESTNO`) vers des messages FR (liste `ERRORS`), et 4 contrôles propres à la caisse (liste `POS_CHECK`, voir plus bas) ; le voyant FDM du bandeau (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/ui/scaffold/StatusLights.kt`, vert / orange / rouge) donne l'état, et `android/app/src/main/kotlin/dev/nuagemagique/pilipos/ui/fdm/FdmBanner` affiche le message sous le bandeau : un bandeau orange pour un avertissement (la vente continue), un voile (`android/app/src/main/kotlin/dev/nuagemagique/pilipos/ui/fdm/FdmBlockingVeil.kt`) qui couvre les écrans de vente pour une erreur bloquante (seul « Réessayer » ; Rapports et Config restent accessibles),

et chaque écran qui reçoit un refus du FDM (encaissement, remboursement, copie, addition, commande, pointage, caisse, rapports) affiche le même message (`FdmTexts.describe`), plus un repli générique pour les erreurs de connexion (mots-clés `connect`/`timeout`) et un repli générique final pour tout code non listé. Le sondage du status FDM tourne dans le moteur, en arrière-plan, toutes les 30 secondes, et s'espace (60 s, 2 min, jusqu'à 5 min) tant que le FDM ne répond pas (`docs/android/ENGINE.md` §7). Les textes ci-dessous sont recopiés **mot pour mot** depuis le code source (source de vérité) ; toute divergence future entre ce tableau et `FdmMessageTables.kt` doit être résolue en faveur du code.

**Avertissements reçus avec une signature :** la description POS v2.2 (Message) demande d'afficher un avertissement au moins une fois par bookingDate, à la réception de la réponse d'une mutation, et une erreur à chaque réception. Chaque réponse de signature vérifiée transmet ses avertissements et informations (sauf `showPos` NEVER) à l'écran (le client FDM de chaque commande porte un relais de messages vers le moteur, qui émet l'événement `fdm-sign-messages`) : un message s'affiche en FR une fois par code et par bookingDate, quelle que soit la commande qui a signé, et le voyant FDM passe à l'orange. Les erreurs sont affichées à chaque refus, comme décrit ci-dessus. Tests : `crates/pos-fiscal/tests/sign_messages.rs`, `android/app/src/test/kotlin/dev/nuagemagique/pilipos/ui/fdm/FdmStoreTest.kt`.

**Essai sur le simulateur module2 :** les quatre conditions de l'onglet Developer du simulateur FDM02000425 ont été rejouées le 27/09/2026 (un status et une vente chacune). BUFFER_FULL et FDM_LOCKED refusent la vente avec leur code, sans consommer de numéro de ticket. BUFFER_NEAR_FULL et TRANSACTION_UPLOAD_FAILED (compteur de tentatives d'envoi) laissent la vente signée, l'avertissement figurant dans le status et dans la réponse de signature. Les réponses sont jointes dans le dossier Report_Testing/module2 (fichiers LIVE_condition_*.json). Les réponses du simulateur (journée complète, refus de jeton, posId inconnu, requête invalide, conditions de buffer) ont aussi été enregistrées une fois : le mock FDM utilisé pour les preuves JSON ci-dessus répond dans la même forme (test `module2_shape.rs`), et le POS accepte chacune de ces réponses (test `module2_recorded.rs`). Un jeton refusé, que module2 signale sans code, affiche le message UNAUTHORIZED.

**Statut tests/captures :** l'injection d'erreurs runtime est implémentée dans le mock FDM (`crates/pos-fdm-mock`, variables d'environnement `MOCK_FDM_ERROR` / `MOCK_FDM_WARNING`, codes séparés par des virgules). Les **preuves JSON** (requête signSale + réponse d'erreur GraphQL, requête status + réponse avec le message) sont capturées par `scripts/capture_fdm_error_evidence.sh` et archivées sous `evidence/fdm_errors/` pour les 4 erreurs à affichage obligatoire (BUFFER_FULL, FDM_LOCKED, FDM_NOT_OPERATIONAL, UNKNOWN_POS), les 2 avertissements à affichage obligatoire (CLIENT_CERT_EXPIRED, BUFFER_NEAR_FULL) et 2 codes optionnels (UNAUTHORIZED, RTC_SYNC_FAILED). Le payload signSale utilisé est la fixture officielle M110 (Basic Set v1.8). Les **captures d'écran** de l'application Android pour chaque message sont dans *CAPTURES_ECRAN_FR.pdf* : captures A56 à A79, A88 à A90 et A93 (profil D3 Pro émulé). Sur une vente, les codes INTERNAL_ERROR, UNDEFINED_ERROR et UNDEFINED_OTHER et une réponse non conforme à la requête (`FDM_RESPONSE_MISMATCH`) laissent l'issue inconnue (le moteur garde la requête et la renvoie à l'identique ; un nouvel essai avec un FDM sain conclut la vente) : l'écran affiche d'abord le message de la boîte noire pour ce code, puis « La vente n'est pas encore enregistrée : le prochain essai envoie à nouveau la même demande à la boîte noire. », voir les captures A65 à A67 et A78. La colonne Capture du tableau ci-dessous donne l'identifiant A de chaque écran.

Tableau de réponse (JSON fournis dans `evidence/fdm_errors/` ; identifiants des captures du profil D3 Pro émulé) :

| Code | Nature | Affichage requis | Affichage pili-pos-android | Capture |
|---|---|---|---|---|
| CLIENT_CERT_NEAR_EXPIRATION | Warning | O | ☒ (mappé « Le certificat de la boîte noire expire bientôt. ») | capture A62 |

| Code | Nature | Affichage requis | Affichage pili-pos-android | Capture |
|---|---|---|---|---|
| CLIENT_CERT_EXPIRED | Warning | M | ☒ (mappé « Le certificat de la boîte noire a expiré. Contactez votre technicien. ») | JSON : `evidence/fdm_errors/CLIENT_CER` ; capture A61 |
| RTC_SYNC_FAILED | Warning | O | ☒ (mappé « L'horloge de la boîte noire n'a pas pu se synchroniser. ») | JSON : `evidence/fdm_errors/RTC_SYNC_F` ; capture A68 |
| UPDATE_URLS_FAILED | Warning | O | ☒ (mappé « Mise à jour des URLs de la boîte noire échouée. ») | capture A69 |
| UPDATE_TRUST_CERTS_FAILED | Warning | O | ☒ (mappé « Mise à jour des certificats de confiance échouée. ») | capture A70 |
| UPDATE_TASK_LIST_FAILED | Warning | O | ☒ (mappé « Téléchargement des tâches FDM échoué. ») | capture A71 |
| TASK_FEEDBACK_FAILED | Warning | O | ☒ (mappé « La boîte noire n'a pas pu signaler le résultat d'une tâche. ») | capture A72 |
| NOP_FAILED | Warning | O | ☒ (mappé « Communication intermédiaire avec la boîte noire échouée. ») | capture A73 |
| BUFFER_NEAR_FULL | Warning | M | ☒ (mappé « La boîte noire est presque pleine. Programmez une intervention. ») | JSON : `evidence/fdm_errors/BUFFER_NEA` ; capture A60 |
| SERVER_CERT_RESOLVE_FAILED | Warning | O | ☒ (mappé « Impossible de résoudre le certificat du serveur FDM. ») | capture A74 |
| TRANSACTION_UPLOAD_FAILED | Warning | O | ☒ (mappé « La boîte noire échoue à transmettre les transactions au SPF Finances. ») | capture A75 |
| INITIALIZATION_FAILED | Warning | O | ☒ (mappé « L'initialisation de la boîte noire a échoué. ») | capture A76 |
| RTC_NOT_INITIALIZED | Warning | N (non-affichable) | - | - |
| CORRUPT_RECORD_ENCOUNTERED | Warning | N | - | - |

| Code | Nature | Affichage requis | Affichage pili-pos-android | Capture |
|---|---|---|---|---|
| UPDATE_PARAMS_FAILED | Warning | N | - | - |
| UPDATE_CLIENT_CERT_FAILED | Warning | N | - | - |
| UPDATE_VAT_RATES_FAILED | Warning | N | - | - |
| UPDATE_POS_ALLOWLIST_FAILED | Warning | N | - | - |
| DUPLICATE_REQUEST | Warning | N | - | - |
| BUFFER_FULL | Erreur | M | ☒ (mappé « La boîte noire est pleine. Contactez votre technicien. ») + modale bloquante au checkout | JSON : `evidence/fdm_errors/BUFFER_FUL` ; capture A56 |
| FDM_LOCKED | Erreur | M | ☒ (mappé « La boîte noire est verrouillée. Contactez votre technicien. ») + modale bloquante | JSON : `evidence/fdm_errors/FDM_LOCKED` ; capture A57 |
| TOO_MANY_MEMORY_ERRORS | Erreur | N | - | - |
| UNAUTHORIZED | Erreur | O | ☒ (mappé « La caisse n’est pas autorisée à parler à la boîte noire. ») | JSON : `evidence/fdm_errors/UNAUTHORIZ` ; capture A63 |
| INVALID_REQUEST | Erreur | O | ☒ (mappé « Erreur de communication avec la boîte noire. Contactez votre technicien. ») | capture A64 |
| INTERNAL_ERROR | Erreur | O | ☒ (sur une vente : « Erreur technique de la boîte noire. Contactez votre technicien. » suivi de « La vente n’est pas encore enregistrée : le prochain essai envoie à nouveau la même demande à la boîte noire. ») | capture A65 |
| UNDEFINED_ERROR | Erreur | O | ☒ (sur une vente : « Erreur indéfinie de la boîte noire. Contactez votre technicien. » suivi de « La vente n’est pas encore enregistrée : le prochain essai envoie à nouveau la même demande à la boîte noire. ») | capture A66 |

| Code | Nature | Affichage requis | Affichage pili-pos-android | Capture |
|---|---|---|---|---|
| UNDEFINED_OTHER | Erreur | O | ☒ (bandeau « Erreur de la boîte noire. Contactez votre technicien. » ; avec le simulateur de la boîte noire, la signature de la vente n'est pas refusée pour ce code) | capture A67 |
| FDM_NOT_OPERATIONAL | Erreur | M | ☒ (mappé « La boîte noire ne fonctionne pas. Impossible d'enregistrer les ventes. ») + modale bloquante | JSON : `evidence/fdm_errors/FDM_NOT_OP` ; capture A58 |
| UNKNOWN_POS | Erreur | M | ☒ (mappé « Cette caisse n'est pas reconnue par la boîte noire. ») | JSON : `evidence/fdm_errors/UNKNOWN_PO` ; capture A59 |

La description détaillée POS v2.2 (`tb_messages`) liste en plus deux avertissements à affichage obligatoire, absents du tableau §V du manuel : `UPDATE_POS_VATNO` (« Le numéro de TVA de la caisse ne correspond pas à celui de la boîte noire. Contactez votre technicien. ») et `UPDATE_POS_ESTNO` (« Le numéro d'unité d'établissement de la caisse ne correspond pas à celui de la boîte noire. Contactez votre technicien. »). Tous deux sont mappés et testés (`android/app/src/test/kotlin/dev/nuagemagique/pilipos/i18n/FdmTextsTest.kt`).

Quatre contrôles propres à la caisse (liste `POS_CHECK` de `FdmMessageTables.kt`, hors tableau §V du manuel, code reconnu dans le texte de l'erreur) ont leur message FR :

| Code POS | Message affiché | Capture |
|---|---|---|
| `FDM_ANSWER_PENDING` | « La caisse attend la réponse de la boîte noire pour l'opération précédente. Vérifiez la connexion et réessayez. » (réponse perdue, sans code de la boîte noire) | capture A79 |
| `FDM_RESPONSE_MISMATCH` | « Réponse de la boîte noire incohérente. Vente non enregistrée. Contactez votre technicien. La vente n'est pas encore enregistrée : le prochain essai envoie à nouveau la même demande à la boîte noire. » (résultat inconnu : la requête est renvoyée à l'identique) | capture A78 |
| `FDM_NOT_INITIALIZED` | « La boîte noire n'est pas encore initialisée. » | capture A90 |
| `FDM_CERTIFICATE_REFUSED` | « Connexion sécurisée à la boîte noire refusée (certificat). Contactez votre technicien. » | capture A93 |

En complément des codes ci-dessus, `FdmMessageTables.kt` prévoit deux messages génériques non listés dans la spec §V : « Connexion perdue avec la boîte noire. Vérifiez le câble réseau. » (si le message brut contient `connect/timeout`) et « Erreur boîte noire. Contactez votre technicien. » (repli final pour tout code FDM non reconnu par le mapping ci-dessus ; l'capture A77 montre la perte de connexion) - ceci garantit qu'aucune erreur FDM ne s'affiche jamais sans message FR compréhensible par l'opérateur.

☒ **Le test a été effectué sur le moteur commun, JSON ajoutés** (`evidence/fdm_errors/`, 8 codes × 4 fichiers, mêmes fichiers que dans le dossier Linux puisque le moteur et le mock FDM sont communs). Captures d'écran de l'application Android : A56 à A79, A88 à A90 et A93.

---

# §VI - TESTING DES TICKETS

## Contenu obligatoire du ticket TVA

Tableau des 18 éléments obligatoires × 6 types de tickets. *Voir : TICKETS_EXEMPLES_FR.md* pour les exemples de tickets et le tableau de conformité coché.

| Élément | N | N Refund | P | C | T | Autres (cuisine) |
|---|---|---|---|---|---|---|
| I. Dénomination complète prestataire | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| II. Identification (nom/adresse/TVA) | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| III. Dénomination commerciale | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| IV. N° unité établissement (EST) | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| V. N° fabrication POS + version | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| VI. Identification terminal | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| VII. Identification dispositif (MAC, SN) | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| VIII. Identification utilisateur (INSZ) | ☒ | ☒ | ☒ | ☒ | ☒ | - |
| IX. Date et heure | ☒ | ☒ | ☒ | ☒ | ☒ | ☒ |
| X. N° ticket continu | ☒ | ☒ | ☒ | ☒ | ☒ | - |

| Élément | N | N Refund | P | C | T | Autres (cuisine) |
|---|---|---|---|---|---|---|
| XI. Opérations produits (qté, desc, prix, TVA) | ☒ | ☒ | ☒ | ☒ | ☒ | (descriptions seules, pas de prix) |
| XII. Total à payer TVA comprise | ☒ | ☒ | - (pas encore payé) | ☒ | ☒ | - |
| XIII. Base imposable par taux | ☒ | ☒ | - | ☒ | ☒ | - |
| XIV. Montant TVA par taux | ☒ | ☒ | - | ☒ | ☒ | - |
| XV. Modifications prix non-lignes | ☐ à la caisse (aucune remise saisie) ; la remise d'une commande en ligne est imprimée sous chaque ligne qu'elle touche | ☐ | ☐ | ☐ | ☐ | - |
| XVI. Données FDM (footer, fdmId, dateTime, label, counters, shortSignature) | ☒ | ☒ | ☒ (sans shortSignature) | ☒ (sans shortSignature) | ☒ (sans shortSignature) | - |
| XVII. Lignes paiement (arrondi, tips, remboursements) | ☒ | ☒ (négatif) | - | - | ☒ (FICTIF) | - |
| XVIII. QR Code URL FDM | ☒ | ☒ | - (sans QR, le FDM ne renvoie pas de verificationUrl pour P) | ☒ | ☒ | - |

**Mentions obligatoires spécifiques :**

- Ticket C (Copy) : **"CECI N'EST PAS UN TICKET DE CAISSE DE TVA VALABLE"** - ☒ imprimé sur tous les tickets C.
- Ticket T (Training) : même mention + bandeau "TRAINING" - ☒.
- Tickets autres (cuisine/bar) : pas de prix, pas de données de contrôle, mention "CECI N'EST PAS UN TICKET DE CAISSE DE TVA VALABLE" - ☒.

## Règles de langue

**Règle appliquée :** conformément au manuel (§VI, p. 99), la langue du ticket est la langue officielle de la région linguistique **où le ticket est émis**, c'est-à-dire celle de l'établissement exploitant le POS. Elle est fixée par installation lors de la mise en service (région de langue française → `fr`, néerlandaise → `nl`, allemande → `de`, Bruxelles-Capitale → `fr` ou `nl`). Détail : *GARANTIE_MAJ_LANGUES_FR.md* §3.2.

**Exemple :** un établissement à 4000 Liège (région de langue française) émet ses tickets en **français**. Le code source supporte aussi le rendu multilingue des libellés produits via la table `translations` (cf. *SCHEMA_BD_FR.md*).

Pour répondre à la spec belge, le POS génère les mentions fiscales du ticket dans la langue configurée au niveau du POS (clé `language` de la configuration, cf. `MANUEL_INSTALLATION_FR.md`), mappée sur une langue nationale via `ReceiptLanguage::from_code` (`crates/pos-fiscal/src/ticket.rs`) : `fr` → FR, `nl` → NL, `de` → DE ; toute langue d'affichage non officielle (zh, en) retombe sur le FR. Les trois jeux de mentions implémentés :

| Mention | FR | NL | DE |
|---|---|---|---|
| Dénomination ticket N | TICKET DE CAISSE T.V.A. | BTW-KASTICKET | MWST.-KASSENZETTEL |
| Remboursement (sous le titre du ticket TVA) | REMBOURSEMENT | TERUGBETALING | RUECKERSTATTUNG |
| Addition (sous PRO FORMA) | ADDITION PROVISOIRE | VOORLOPIGE REKENING | VORLAEUFIGE RECHNUNG |
| Rapport X / Z | X / Z CHIFFRE D'AFFAIRES | X / Z OMZET | X / Z UMSATZ |
| Mention d'invalidité (bas de ticket non-N) | CECI N'EST PAS UN TICKET DE CAISSE T.V.A. VALABLE | DIT IS GEEN GELDIG BTW-KASTICKET | DIES IST KEIN GUELTIGER MWST.-KASSENZETTEL |
| Copie | COPIE | KOPIE | KOPIE |
| Facture | FACTURE | FACTUUR | RECHNUNG |

Les libellés FR sont ceux de la loi : arrêté royal du 30 décembre 2009, art. 2 (« TICKET DE CAISSE T.V.A. ») et arrêté ministériel du 29 avril 2024, art. 20, 29, 30 et 38 (« CECI N'EST PAS UN TICKET DE CAISSE T.V.A. VALABLE », « ADDITION PROVISOIRE », « PRO FORMA », « Z CHIFFRE D'AFFAIRES ») ; les autres, de la description détaillée POS v2.2 et du tableau §VI du manuel ; les libellés NL des circulaires GKS ; les libellés DE sont une traduction de style officiel (translittération ASCII, l'imprimante thermique n'ayant pas de négociation de code page) ; les documents du SPF fournis ne contiennent pas de texte de référence allemand, ces libellés ne sont donc pas validés contre une source officielle. La langue est un paramètre d'installation (`pos.json::language`) : aucun changement de binaire d'un établissement à l'autre.

**Cette politique de langue par client est documentée dans `MANUEL_INSTALLATION_FR.md` §"Langue du ticket".**

## Contenu obligatoire de la facture imprimée (B2B)

*Voir : DECISION_PEPPOL_AP_FR.md + FLUX_REMBOURSEMENT_FR.md* §3. L'émission de factures B2B (event I) est implémentée de bout en bout côté code : `invoice::do_sign_invoice_fiscal` construit l'InvoiceInput (n° de facture séquentiel, TVA client, `fdmRefs` vers les ventes sous-jacentes - forme de la fixture M150), l'envoie au FDM via `signInvoice` et archive l'événement I - exercé en e2e (`spf_e2e_basic::m150_*`) sous les features par défaut (`hubrise` + `peppol`). Mais ce code est **hors périmètre de la build cert v1.0.0** : la fonction `cert` du moteur (`cert = ["hubrise"]`) n'active pas `peppol`, donc `invoice.rs` n'est pas compilé dans la bibliothèque `libpos_ffi.so` soumise. **Aucune facture B2B n'est émise ni imprimée par le POS au rendez-vous d'admission.** L'UI complète + le câblage AP Peppol sont planifiés pour une mise à jour certifiée ultérieure.

Les 11 éléments obligatoires (a-k) sur la facture seront couverts et cochés au dossier de la mise à jour correspondante, une fois le feature flag `peppol` activé dans une build cert et le flux UI complet livré.

---

# §VII - ACQUISITION DES DONNÉES

## §VII.A Les données sont conservées localement

*Voir : FEUILLE_ACQUISITION_DONNEES_FR.md* §1 et §2.2 et *SECURITE_BD_FR.md* §8 - procédure complète d'accès (export SPF par le Gérant sur une clé USB ou une carte SD), copie et extraction des données du fichier SQLite `pos.db`.

## §VII.B Les données sont conservées dans le cloud

☒ **J'ai pris ceci en considération et j'ai fourni une procédure à l'équipe SCE** (cochée par défaut bien que non applicable : les données fiscales légales de `pili-pos-android` restent sur la caisse et aucune donnée fiscale n'est envoyée par défaut ; voir la précision ci-dessous sur l'export nocturne facultatif. La déclaration cochée reconnaît qu'on a pris en compte la question).

**Précision sur l'export nocturne (facultatif).** Les données légales (le journal `fiscal_events`, signé par le FDM) restent **sur la caisse**, dans `pos.db` ; la procédure d'accès du §VII.A est inchangée. Quand la section `export` de `pos.json` est renseignée (absente par défaut, donc absente sur la machine de test), une **copie chiffrée** de la base et une copie statistique nettoyée peuvent être envoyées, après chaque rapport Z réel, au point de réception propre à l'installation. La sauvegarde est chiffrée de bout en bout (age, X25519) pour le propriétaire de l'établissement : le serveur de réception ne détient la sauvegarde que sous forme chiffrée et ne peut pas la lire (il reçoit aussi la copie statistique, non chiffrée, sans donnée directement identifiante, opérateurs sous pseudonyme). Il s'agit d'une copie de sauvegarde et de suivi statistique : elle n'est ni la source des tickets, ni une condition de la vente, et **ne fait pas du POS un POS dans le cloud** (la base reste locale, la signature est faite par le FDM sur le réseau local, la caisse fonctionne sans Internet). La sauvegarde complète ne quitte la caisse que chiffrée ; la copie statistique (montants de vente, de TVA et de paiement par ticket et par ligne, sans donnée directement identifiante) part en HTTPS et peut être lue par le récepteur de l'installation. Détail : *EXPORT_NOCTURNE_FR.md*.

## Feuille d'acquisition des données (DAS)

*Voir : FEUILLE_ACQUISITION_DONNEES_FR.md* qui couvre l'intégralité des §A-F (Informations générales, Réseaux & Appareils, Logs, Rapports X/Z, Base de données, Version du logiciel).

Recap des réponses clés DAS (extrait, le détail est dans l'annexe) :

| Champ | Valeur |
|---|---|
| Producteur | Bruno Moyse / nuagemagique.dev |
| Point de contact | Bruno Moyse |
| E-mail | cloud@nuagemagique.dev |
| Nom du logiciel | pili-pos-android |
| N° de version | 1.0.0 |
| Système d'exploitation | Android 13 (API 33) sur Sunmi D3 Pro (variante avec services Google), application en mode kiosque (Lock Task) |
| Numéro de certificat | (à compléter par le SPF Finances) |
| Date de certification | (à compléter par le SPF Finances) |
| Agent de certification | (à compléter par le SPF Finances) |
| SCE basé sur un système Cloud | ☐ Non |

| Champ | Valeur |
|---|---|
| Connexion internet requise | ☐ Non par défaut (le POS parle au FDM sur le LAN ; seul le FDM nécessite Internet). Sorties HTTPS (443) optionnelles, seulement si activées : `api.scaleway.com` (ticket numérique), `api.hubrise.com` et `manager.hubrise.com` (commandes en ligne, `hubrise_enabled`), point de réception propre à l'installation (export nocturne, section `export` de `pos.json`), registre public des artefacts déclarés (mise à jour lancée par un Gérant) - §III.B |
| Connexion avec des logiciels externes (commande en ligne) | ☒ Oui : HubRise, coupé par défaut (`hubrise_enabled`, activé par installation par un Gérant) ; *COMMANDES_EN_LIGNE_FR.md* |
| Connexion avec un logiciel interne (statistiques et sauvegarde) | ☒ Oui, facultative et absente par défaut : l'export nocturne (section `export` de `pos.json`) envoie après chaque rapport Z réel une sauvegarde chiffrée de bout en bout et une copie statistique au point de réception propre à l'installation. Les données légales restent locales ; *EXPORT_NOCTURNE_FR.md* |
| Connexion POS-FDM | ☒ Réseau local (Wi-Fi, ou liaison filaire si le modèle du terminal en offre une : À effectuer (EV-15 : liaison filaire d'un vrai D3 Pro)) |
| Périphériques supplémentaires | ☐ Non |
| Logs disponibles, lisibles, copiables | ☒ Oui (fichiers `files/logs/pili-pos.log.AAAA-MM-JJ` + table `fiscal_events`, repris par l'export SPF) |
| Méthode de lecture des logs | Export SPF par le Gérant (dossier `logs/` et copie de `pos.db`), puis lecture de la copie sur l'ordinateur de l'agent, par exemple avec `sqlite3` et `.dump fiscal_events` ; *FEUILLE_ACQUISITION_DONNEES_FR.md* §1.3, §2.2 et §2.3 |
| Logiciel DBMS | SQLite 3.45+ (embarqué) |
| Langage de programmation | Rust + Kotlin |
| Sécurité base de données | *Voir : SECURITE_BD_FR.md* |
| Login SPF Finances | Aucun compte système ; profil Gérant de la machine de test (§III) pour lancer l'export |
| Mot de passe SPF Finances | PIN du profil Gérant de la machine de test (§III), communiqué à l'Intake |
| Version du logiciel | 1.0.0 (versionCode 10000), date : À effectuer (EV-18 : date de dépôt), SHA-1 de l'APK signé : À effectuer (EV-01 : SHA-1 et SHA-256 de l'APK signé) |

## §VII.D Rapports X/Z

**Comment les rapports financiers X/Z sont-ils visibles/imprimables ?**

Le Gérant (rôle Admin) ouvre l'écran "Rapports" par le bouton de l'en-tête (*PROFILS_UTILISATEURS_FR.md* §3). Deux boutons distincts "Rapport X (intermédiaire)" et "Rapport Z (clôture)". Le Z est normalement fait par le Gérant ; si une journée reste ouverte sans Z, la caisse fait elle-même le Z, en filet de sécurité (§III.L, FAQ Q501c). Le clic déclenche : 1. La commande du moteur `commands::report::do_generate_x_report` ou `do_generate_z_report`. 2. L'envoi au FDM via `signReportTurnoverX/Z` (et `signReportUserX/Z` pour Z atomique). 3. Sur succès : impression automatique du ticket de rapport sur l'imprimante thermique. 4. Affichage UI d'un récapitulatif lisible (montants par TVA, par moyen de paiement, par utilisateur).

*Voir : MANUEL_UTILISATEUR_FR.md §10 “Rapports X et Z” et TICKETS_EXEMPLES_FR.md §6 (rapport X) et §7 (rapport Z) pour le rendu papier.*

**Captures d’écran à fournir** : oui, *CAPTURES_ECRAN_FR.pdf* (écrans 14 et 15 du dossier Linux, repris dans le PDF Android : captures A36 à A40, rapports X et Z).

**Comment les rapports d’utilisateur X/Z sont-ils visibles/imprimables ?**

Émis automatiquement avec le rapport financier Z dans la même opération atomique M184 (un seul appel GraphQL aliase `signReportTurnoverZ` + `signReportUserZ`). Le rapport utilisateur Z imprime un deuxième ticket séquentiellement après le rapport financier. Le rapport utilisateur X (M182) est déclenché séparément depuis le menu Rapports si l’opérateur le demande.

*Voir : COUVERTURE_RAPPORTS_FR.md §M184 pour le détail de l’émission combinée.*

---

# §VIII - SIGNATURE

| Item | Réponse |
|---|---|
| ☒ Demande complète et signée | Sera cochée et signée électroniquement au moment de la soumission par Bruno Moyse via certificat eID belge ou Itsme. |
| ☒ Résultats des tests présents dans le PDF ou dans les annexes | *Voir : RESULTATS_TESTS_UC_FR.pdf + RAPPORT_TESTS_BASELINE_FR.md + COUVERTURE_FIXTURES_FR.md + COUVERTURE_RAPPORTS_FR.md + COUVERTURE_ADVANCED_SET_FR.md*. |

**SIGNEZ ICI :** signature électronique qualifiée apposée sur le PDF final du formulaire au moment de l’envoi à `secr.gksce@minfin.fed.be`.

---

# ANNEXES DU DOSSIER

Liste des fichiers du ZIP de soumission de `pili-pos-android` (la liste exacte, fichier par fichier, est `MANIFEST.txt`) :

Dans le texte des annexes, un renvoi à un fichier `.md` (par exemple *SECURITE_BD_FR.md*) désigne l’annexe PDF du même nom. Une annexe marquée « commun » décrit le moteur seul : c’est celle du dossier Linux au même état du moteur (étiquette de gel `linux-v1.0.0-rc2`, puis étiquette finale), reprise sans modification (*INDEX_DOSSIER_FR.md* §2 et §3).

| # | Fichier | Section couverte |
|---|---|---|
| 0 | INDEX_DOSSIER_FR.pdf | liste des documents du dossier (article 101), lecture des documents communs |
| 1 | REPONSES_MANUEL_CERT_FR.pdf (ce document) | parcours linéaire §I-VIII |
| 2 | LISTE_DISTRIBUTEURS_FR.pdf | §I.E |
| 3 | PROFIL_ENTREPRISE_FR.pdf | §I.F |
| 4 | MANUEL_UTILISATEUR_FR.pdf | §I.C |
| 5 | MANUEL_INSTALLATION_FR.pdf | §I.C |

| # | Fichier | Section couverte |
|---|---|---|
| 6 | MANUEL_PROGRAMMATION_FR.pdf | §I.C |
| 7 | DECLARATION_FONCTIONS_INTERDITES_FR.pdf | §III.G |
| 8 | PROFILS_UTILISATEURS_FR.pdf | §III.C |
| 9 | DOCUMENT_PROCESSUS_TECHNIQUE_FR.pdf | §II + §III.E |
| 10 | FLUX_DONNEES_FR.pdf | §II.C + §III |
| 11 | SCHEMA_BD_FR.pdf (commun) | §II.C |
| 12 | SECURITE_BD_FR.pdf | §II.C, accès du SPF aux données (articles 16 et 102) |
| 13 | SCHEMA_ER_FR.pdf (commun) | §II.C |
| 14 | SCHEMA_ASSEMBLAGE_FR.pdf | §I.C + §II.G |
| 15 | LISTE_FONCTIONNALITES_FR.pdf | §III |
| 16 | LIGNES_PRODUITS_FR.pdf | §III, Lignes de produit |
| 17 | STRATEGIE_COMPTEURS_FR.pdf (commun) | §III.B (numérotation) + Connexion avec le FDM C (FDM non-opérationnel) |
| 18 | FLUX_REMBOURSEMENT_FR.pdf | §III.F |
| 19 | TICKETS_EXEMPLES_FR.pdf | §VI |
| 20 | DECISION_PEPPOL_AP_FR.pdf | §III.F (encadré B2B) |
| 21 | MAPPING_CONFORMITE_FR.pdf | §IV + §V |
| 22 | FEUILLE_ACQUISITION_DONNEES_FR.pdf | §VI |
| 23 | RAPPORT_TESTS_BASELINE_FR.pdf | §IV + §VIII |
| 24 | COUVERTURE_FIXTURES_FR.pdf (commun) | §IV |
| 25 | COUVERTURE_RAPPORTS_FR.pdf (commun) | §IV |
| 26 | COUVERTURE_ADVANCED_SET_FR.pdf | §IV |
| 27 | RESULTATS_TESTS_UC_FR.pdf | §IV - PDF de résultats UC |
| 28 | CAPTURES_ECRAN_FR.pdf | §V + §VII.D |
| 29 | JOURNAL_TESTS_PRE_SOUMISSION_FR.pdf | §V (répétition pré-soumission) |
| 30 | CHECKLIST_DEPLOIEMENT_FR.pdf | machine de test |
| 31 | GARANTIE_MAJ_LANGUES_FR.pdf | §I.D + §II.D + §II.E + §VI (langue du ticket) |
| 32 | TICKET_NUMERIQUE_FR.pdf (commun) | §III.H |
| 33 | COMMANDES_EN_LIGNE_FR.pdf | §III.M + livraison, commandes en ligne |
| 34 | EXPORT_NOCTURNE_FR.pdf | Configuration Internet + §VII.B + DAS (export nocturne facultatif) |
| 35 | CADRE_DEMANDE_FR.pdf | cadrage de la demande (article 99), versions, moteur commun |
| 36 | PLATEFORME_ANDROID_FR.pdf | §I.C, §II.C, §II.F, §II.G : plateforme, verrouillage, identifiant de l'appareil, stockage |

Plus : - `20250115_FORMULAIRE DEMANDE_CERTIF_POS-FDM_signe.pdf` - formulaire SPF rempli et signé. - Preuves brutes : `evidence/` (Report_Testing dans le ZIP : JSON requête/réponse par UC, tickets imprimés, rapports de construction ; *INDEX_DOSSIER_FR.md* §4). - Hash SHA-1 de l'APK signé `pili-pos-android` : §I.D. - Machine de test : profil émulé du D3 Pro (ou D3 Pro réel : À effectuer (EV-15 : faits d'un D3 Pro réel, ou déclaration du profil émulé)) et simulateur FDM en ligne, voir *CADRE_DEMANDE_FR.md* §8.